

MACHINE LEARNING
@ BERKELEY

Physics ML Seminar 0

Chris John, Avyukth Harish, and Sahir Tandon

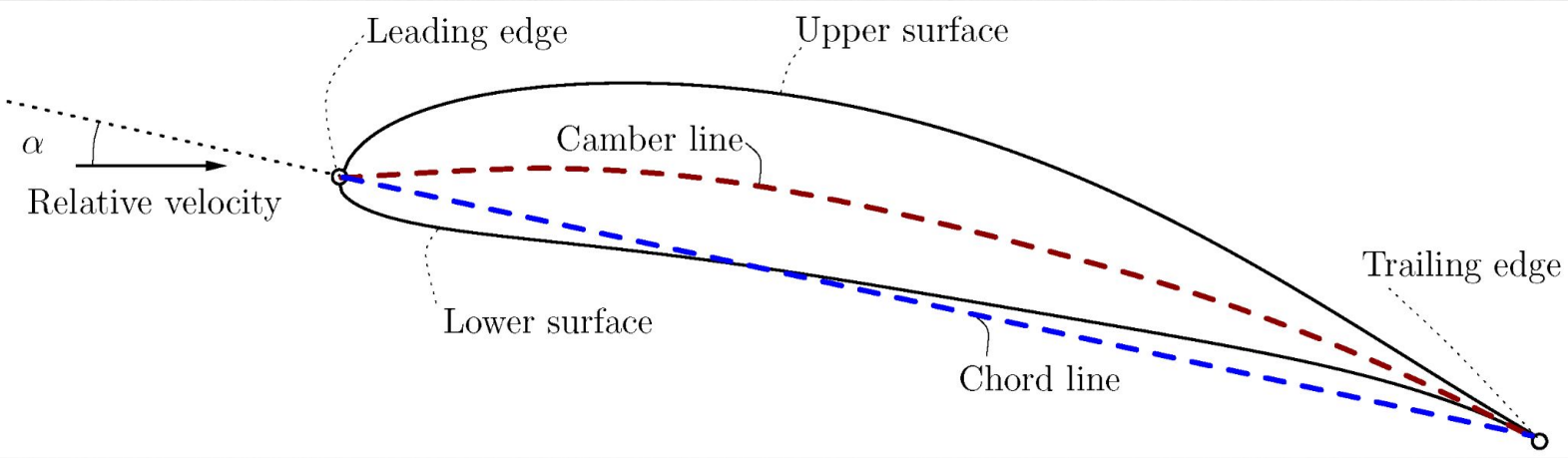
01

What is Physics ML?

What if?

MACHINE LEARNING
@ BERKELEY

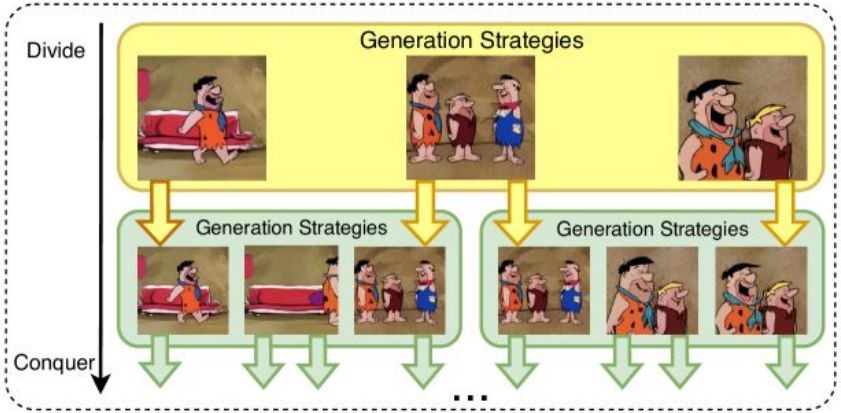
You're Boeing.

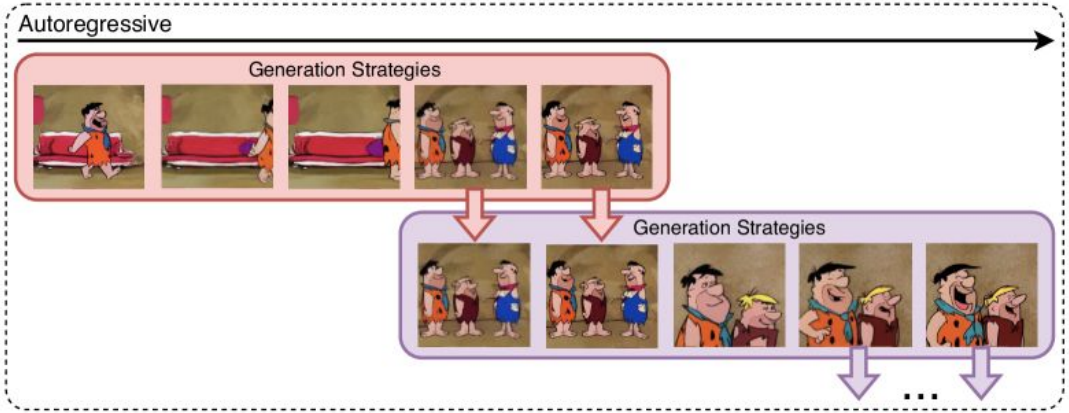


What if?

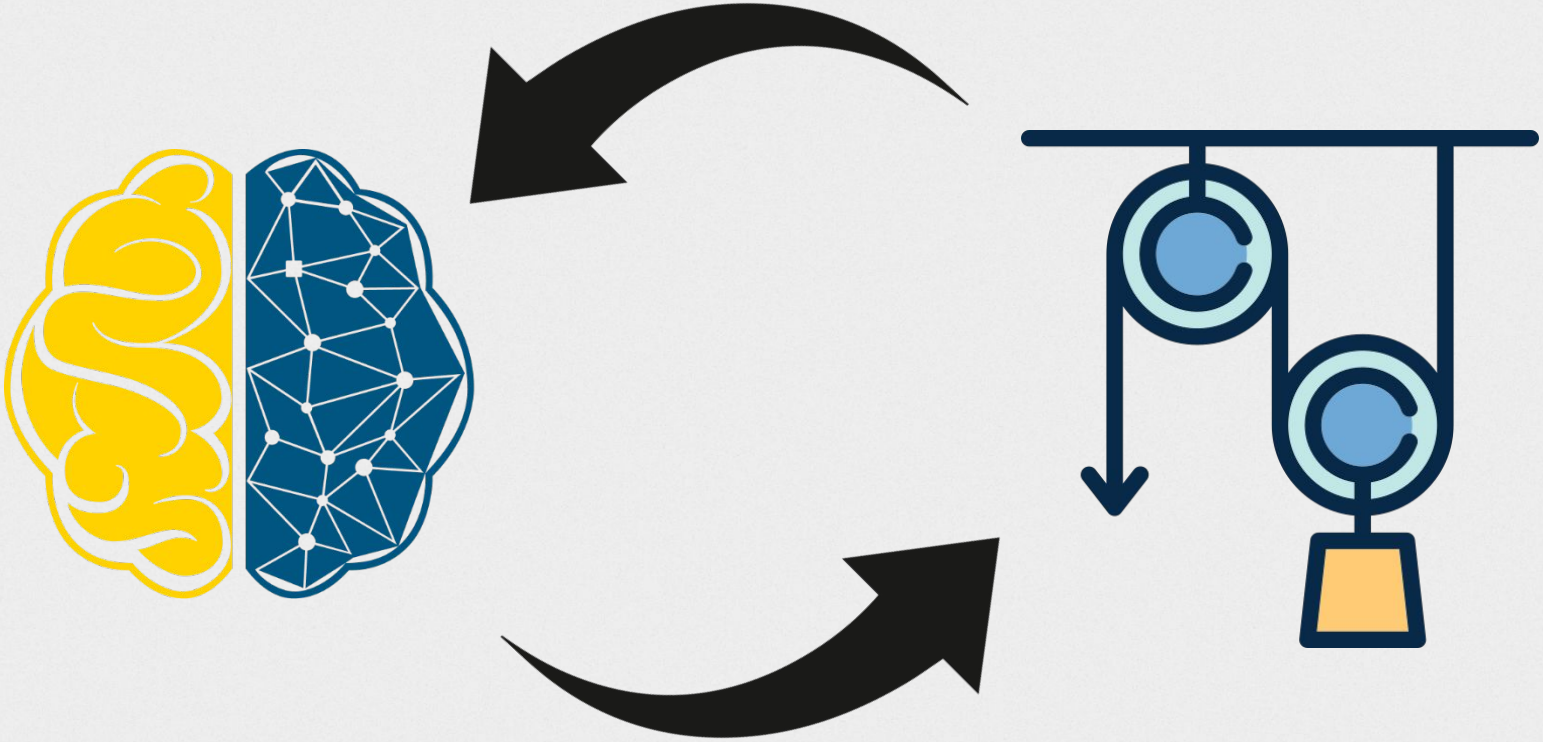
MACHINE LEARNING
@ BERKELEY

You're an ML researcher.





2 Directions of Physics ML



2 (general) Approaches

MACHINE LEARNING
@ BERKELEY

Additional measurement data (optional)

$$L_{Data}(\sigma) = \frac{1}{N}\sum_{i=1}^{N}\left|\theta_{pred}(t_i;\sigma) - \theta_{meas}(t_i)\right|^2$$

+

Boundary/Initial Conditions (optional)

$$L_{Conds}(\sigma) = \frac{1}{2}\left|\theta_{pred}(t_0;\sigma) - \theta_0\right|^2 + \frac{1}{2}\left|\frac{\partial}{\partial t}\theta_{pred}(t_0;\sigma) - v_0\right|^2$$

+

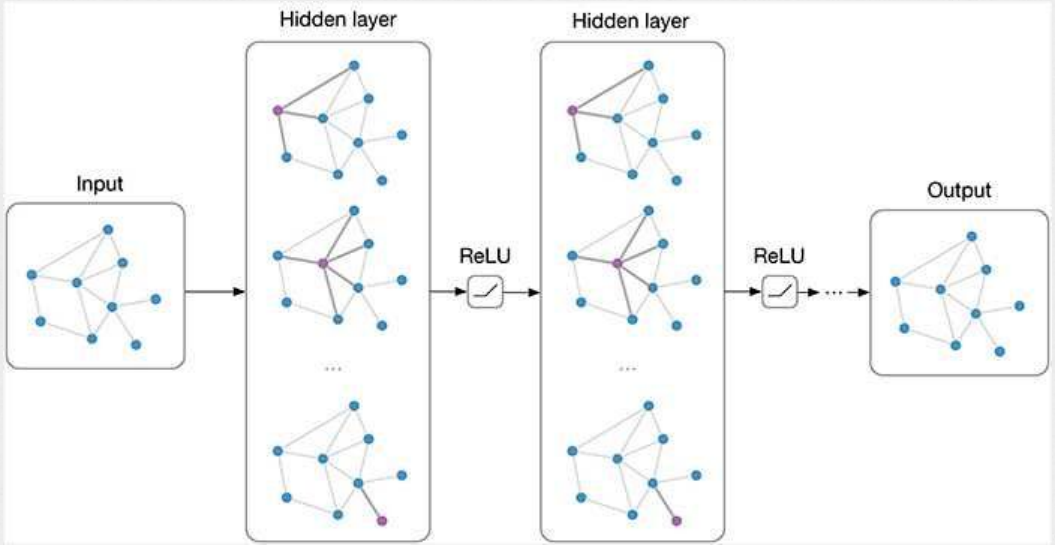
Differential Equation

$$L_{Physics}(\sigma) = \frac{1}{M}\sum_{i=1}^{M}\left|\frac{\partial^2}{\partial t^2}\theta_{pred}(t_i;\sigma) + 2\beta\frac{\partial}{\partial t}\theta_{pred}(t_i;\sigma) + \omega_0^2\theta_{pred}(t_i;\sigma)\right|^2$$

⇩

Loss function

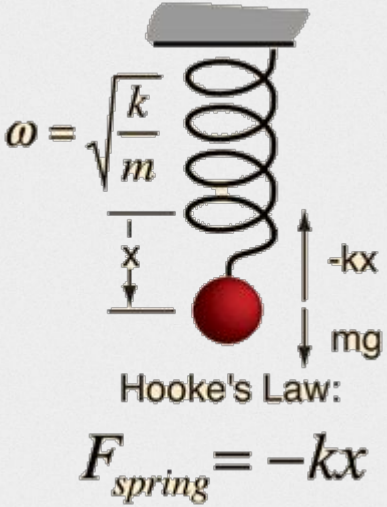
$$L(\sigma) = L_{Physics}(\sigma) + L_{Conds}(\sigma) + L_{Data}(\sigma)$$

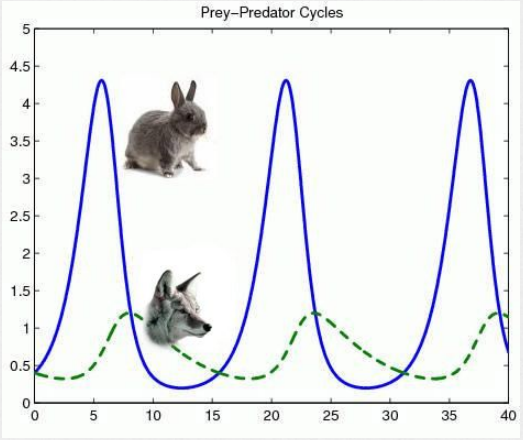


Background: What are Differential Equations?

MACHINE LEARNING @ BERKELEY

- Newton's Second Law: F=ma
- Differential Equations: equations with derivatives
  - Model growth/change/motion
- Highly informative in modeling systems
- **Quickly get hard to solve**

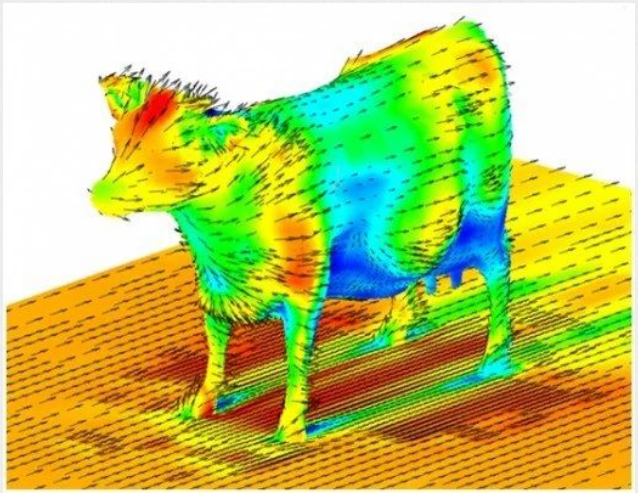


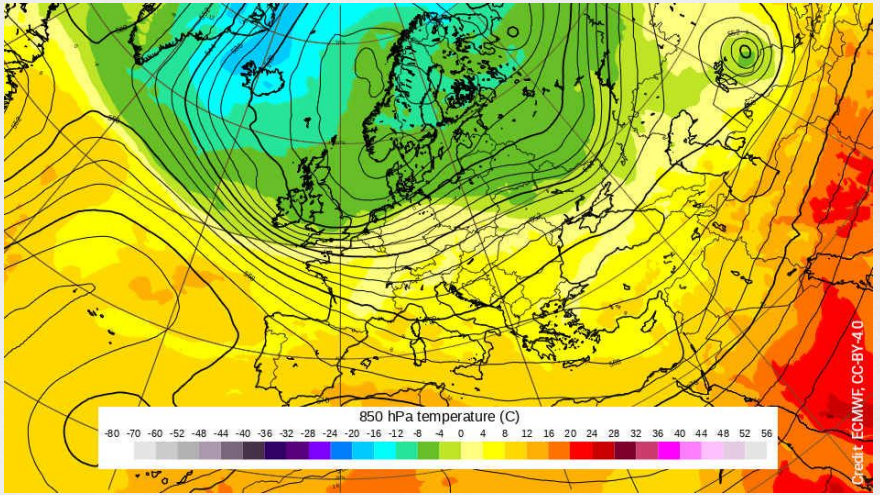


Background: What are PDEs?

MACHINE LEARNING
@ BERKELEY

- Partial Differential Equations (depend on more than 1 variable)



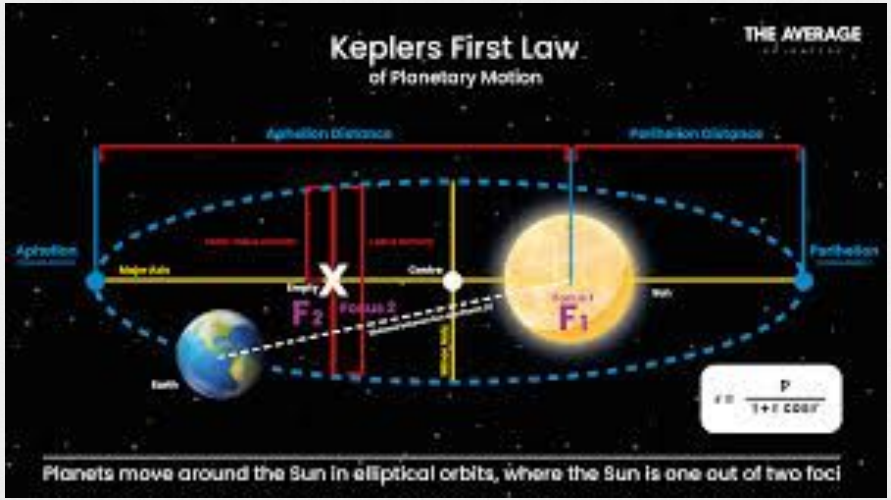


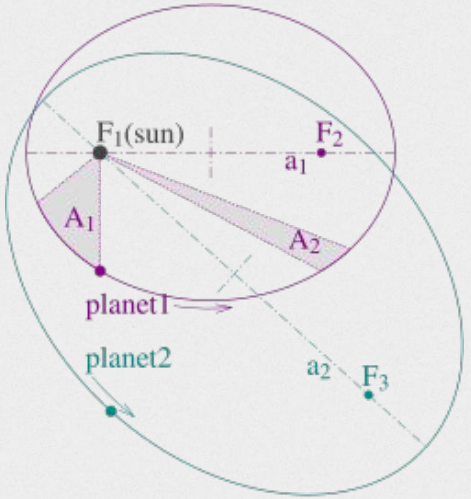
02

Physics-informed Neural Networks

Physics ML: Why is it important?

- Kepler: data-driven model of planetary motion -> elliptical orbit
- Newton: dynamic relationship between momentum and energy

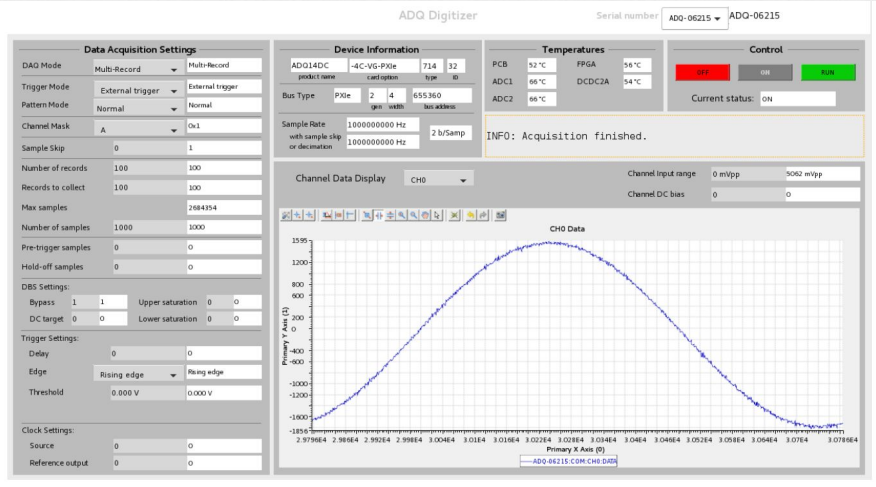




Physics ML: Motivation

- Cost of data acquisition can be high
- SoTA ML techniques lack robustness with these tasks





Physics ML: Motivation

MACHINE LEARNING
@ BERKELEY

- **Traditional ML:** Fit a model to some data from the world
- **Cons**
  - Data can be expensive to gather
  - Your instruments may be limited in the precision it can gather data with

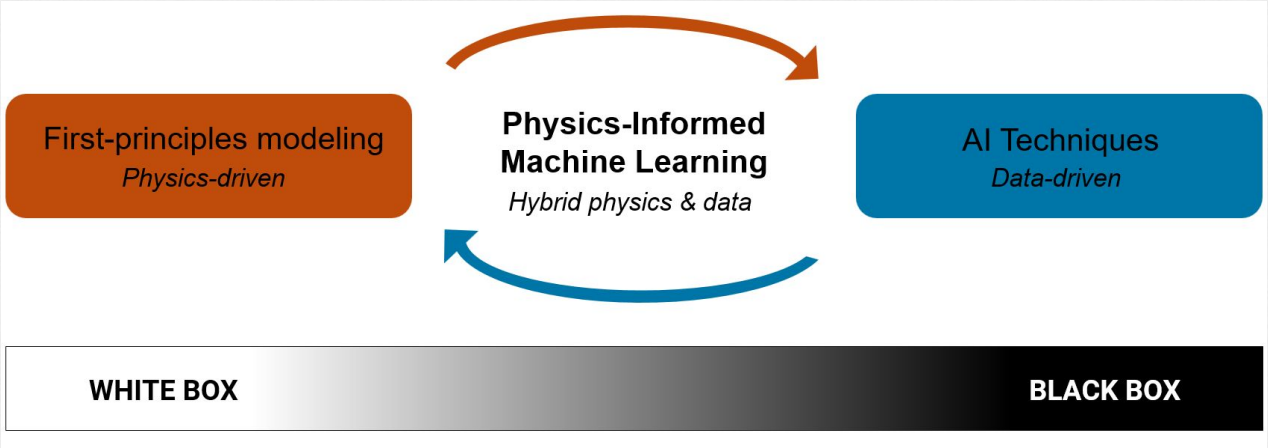




Physics ML: Motivation

MACHINE LEARNING @ BERKELEY

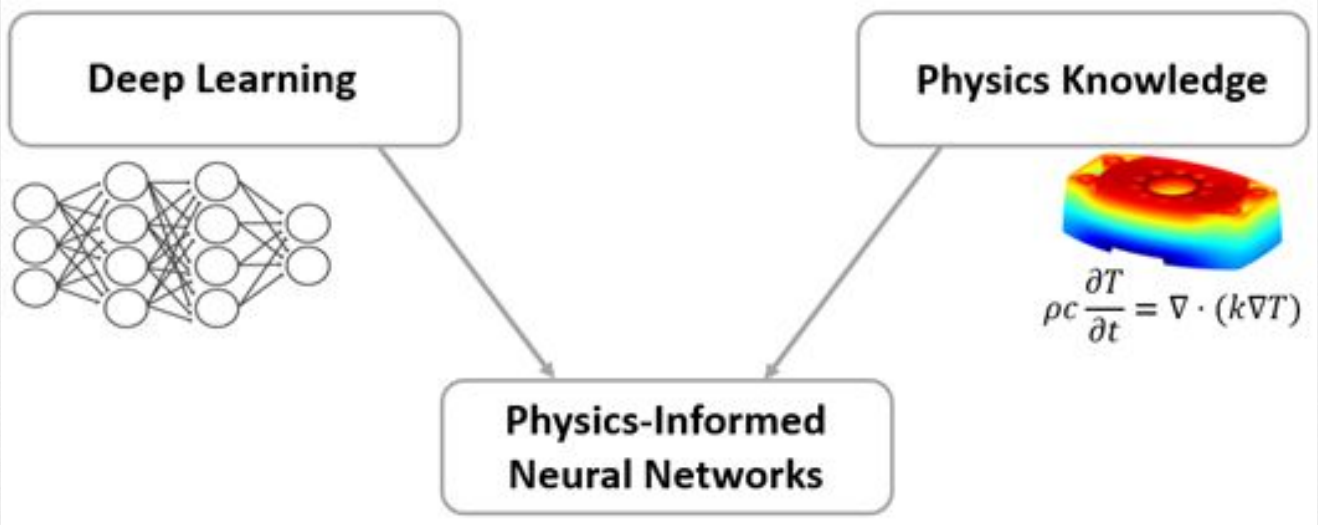
- Vast amount of prior knowledge on physical systems not being utilized
- Is there a way to encode this information to “steer” ML models to obtain a better optimum more efficiently?



PINNs: Overview

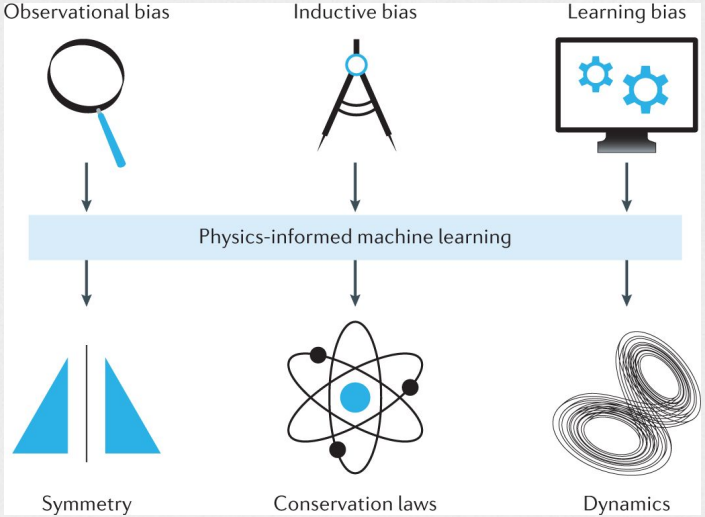
MACHINE LEARNING
@ BERKELEY

- In particular, can we integrate physics into neural networks?



PINNs: What are they?

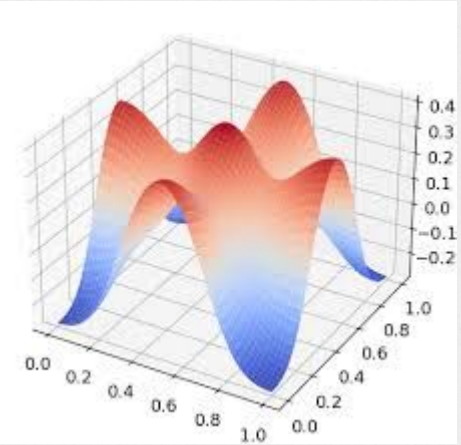
- NNs constrained to respect any symmetry, invariance, or conservation principles originating from the physical laws that govern observed data
- Modeled by general time-dependent and nonlinear differential equations.

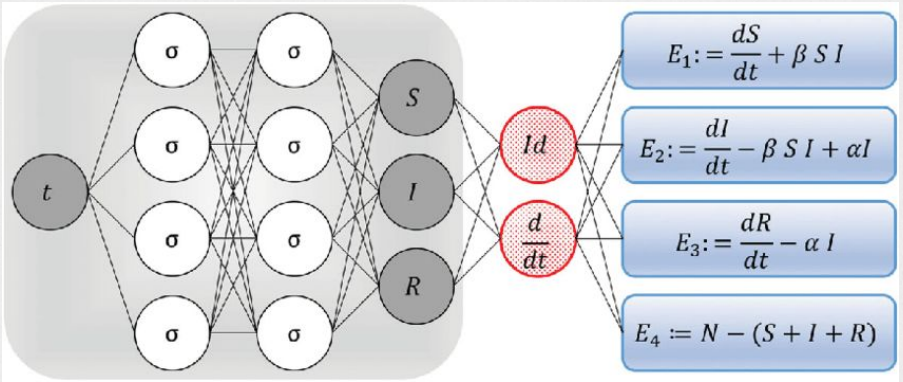


PINNs: What are they?

MACHINE LEARNING @ BERKELEY

- **Key idea:** Additional physics knowledge regularizes the predicted solution in the presence of noisy measurement data, enabling PINNs to learn the true underlying signal rather than overfitting the noisy data
- We can embed this knowledge in the loss function for the model!

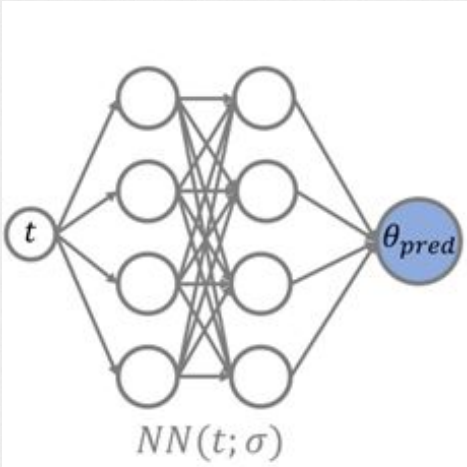


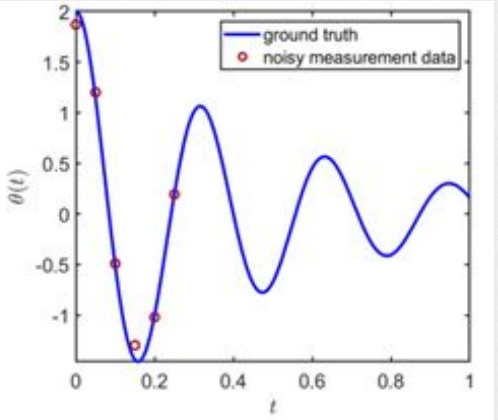


PINNs: Example

MACHINE LEARNING @ BERKELEY

- Imagine you collect some noisy values of a damped pendulum, and want to fit some model to it.
  - Note: This is a toy example. Normally we can't easily obtain an exact solution to fit!
- The normal ML way to do this is to train a model with an MSE loss.



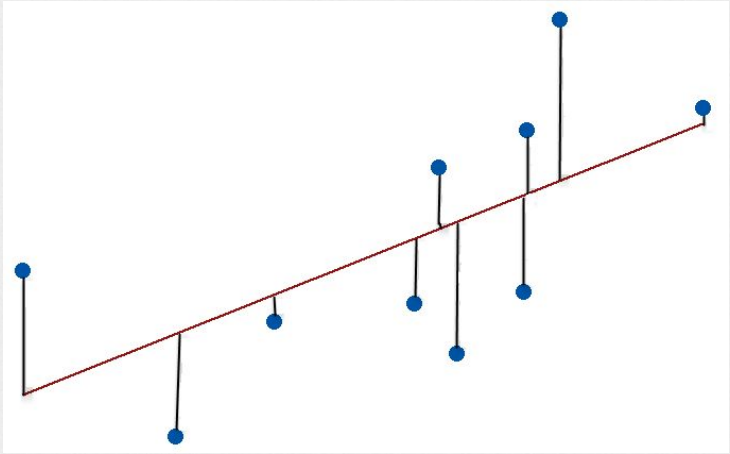


$$\min_{\sigma} \frac{1}{N} \sum_{i=1}^{N} \left| \theta_{pred}(t_i;\sigma) - \theta_{meas}(t_i) \right|^2$$

PINNs: How do they work?

MACHINE LEARNING
@ BERKELEY

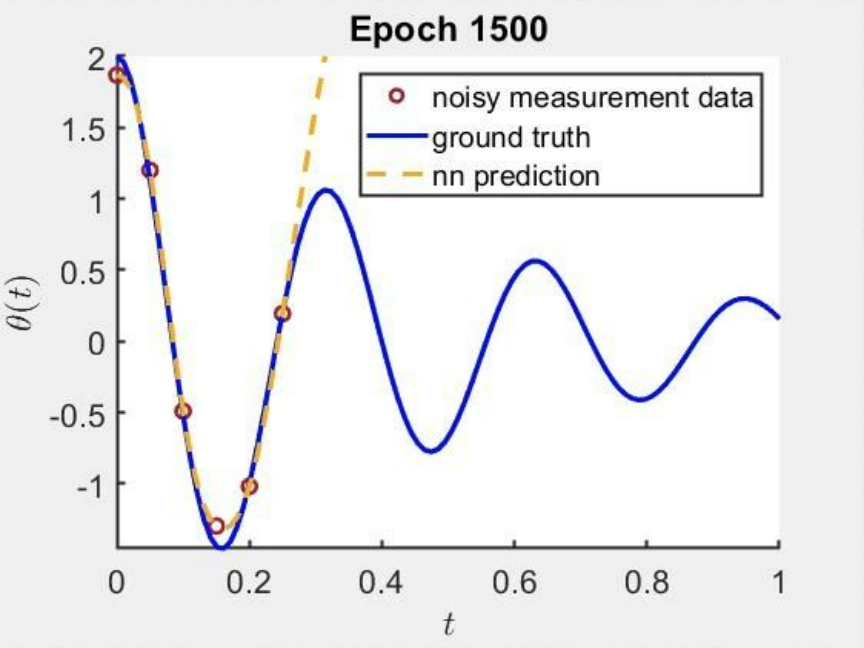
- Review: How is the MSE loss term computed?
  1. Take your labeled points (t, x(t)), and run it through the model to get (t, y(t)), where y is the output from the model.
  2. For each point, compute (y-x)**2, and take the mean of the squared error. This is the loss!



PINNs: How do they work?

MACHINE LEARNING
@ BERKELEY

- NN struggles to accurately make predictions of the system outside training data



PINNs: How do they work?

MACHINE LEARNING
@ BERKELEY

- Acquiring more data COULD enhance predictions but…
  - Most of the time is expensive or impossible!
- System is modeled by a damped pendulum
  - Approximated by linear, second-order diff eq for small angles

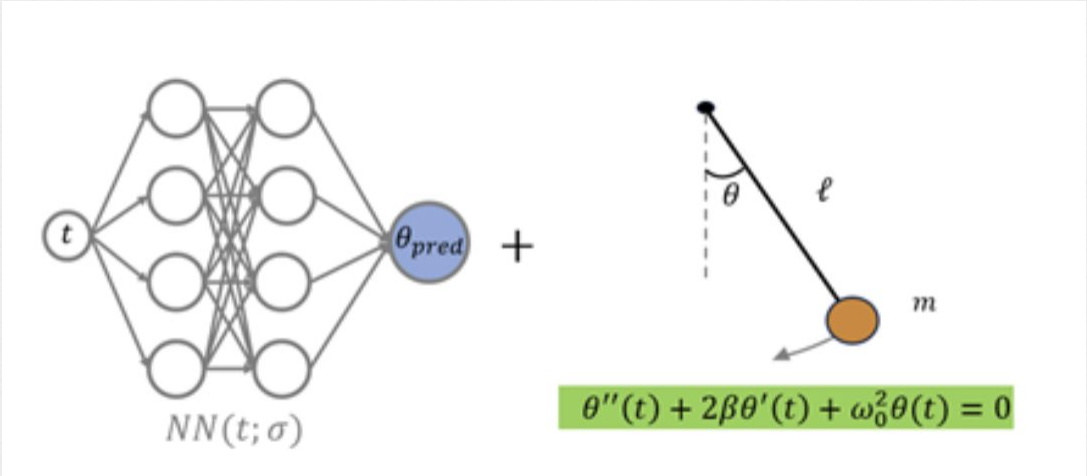
$$\theta''(t) + 2\beta\theta'(t) + \omega_0^2\theta(t) = 0$$

PINNs: How do they work?

MACHINE LEARNING
@ BERKELEY

$$\theta''(t) + 2\beta\theta'(t) + \omega_0^2\theta(t) = 0$$

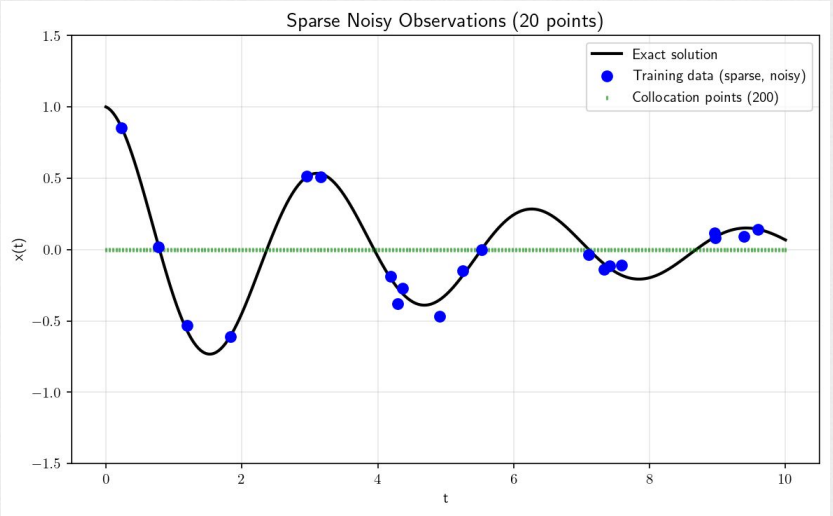
- Ordinarily, this knowledge would be ignored
- PINNs incorporate diff eqs as an additional, physics-informed term IN the loss function



PINNs: How do they work?

MACHINE LEARNING
@ BERKELEY

- Add a physics loss term!
  1. **For each epoch, create a grid of N different sample time points, t** - these are called collocation points.
  2. **Run each sample time through the model,** to get N points (t, **θ**(t)).
     i. Note: We can sample as many points as we want from the model, even if it's outside of our original data!



Why is this useful? We don't have known data from outside of the domain of the sample

PINNs: How do they work?

MACHINE LEARNING
@ BERKELEY

$$\theta''(t) + 2\beta\theta'(t) + \omega_0^2\theta(t) = 0$$

- How do we compute the physics loss (continued)?

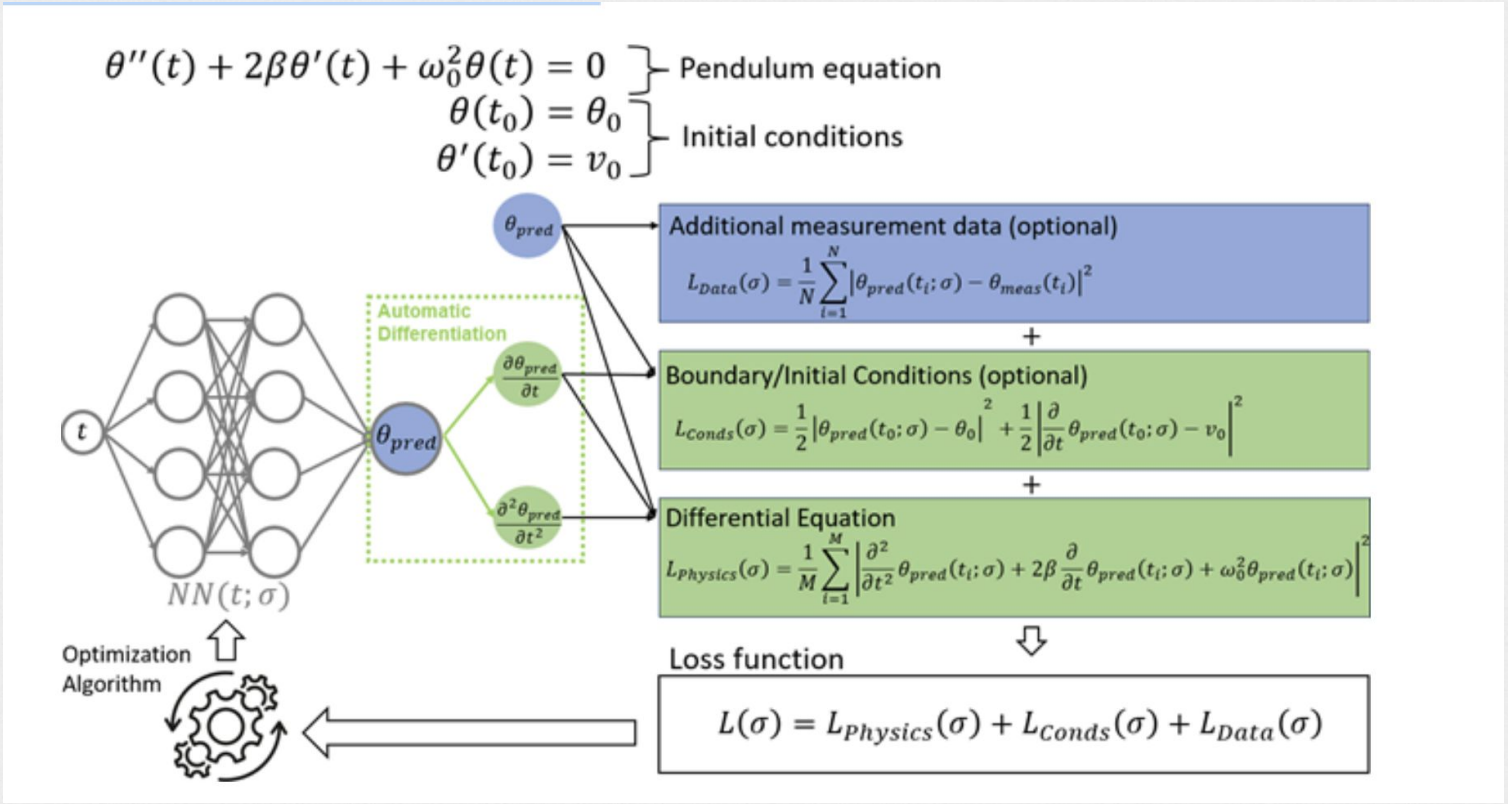
  3. Compute the first and second time derivatives of θ(t), for our collocation points.

  i. Compute the first and second derivative of the output **θ** with respect to the input **t**

  4. Use our derivatives to evaluate the LHS of the differential equation.

  5. Since we know the LHS of the equation should be 0 ideally, we can **compute the MSE of all of our evaluated values** for the LHS. This is the loss!

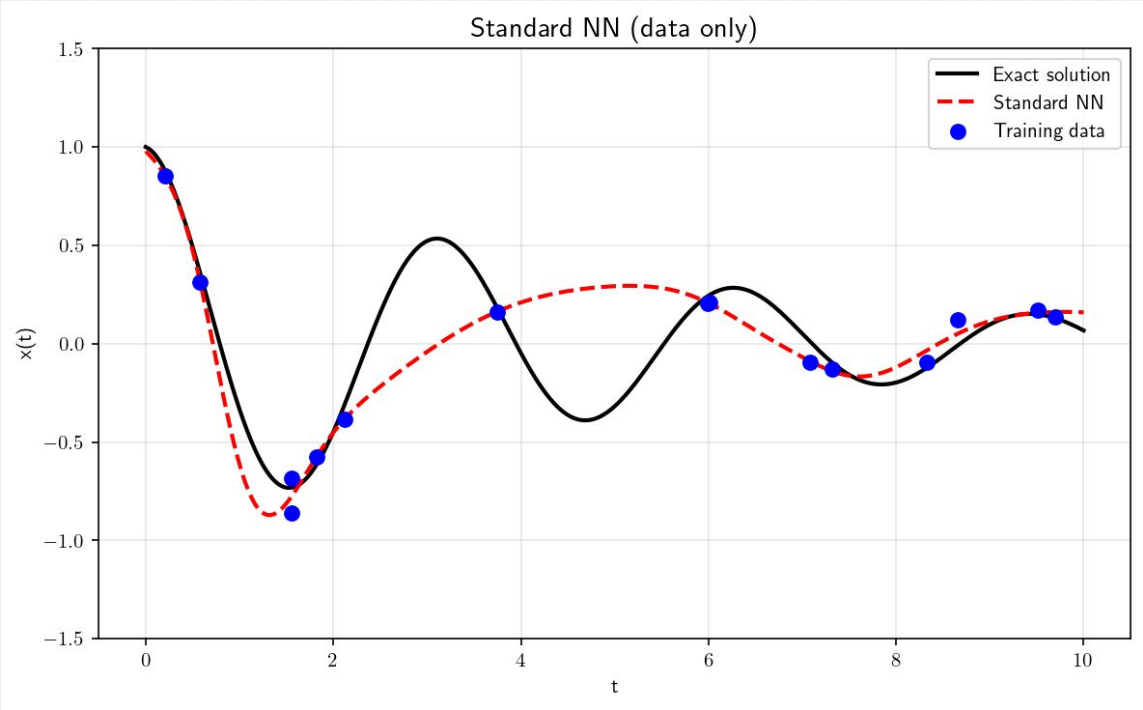
PINNs: How does the loss function work?



PINNs: Example

- With a normal FFN:

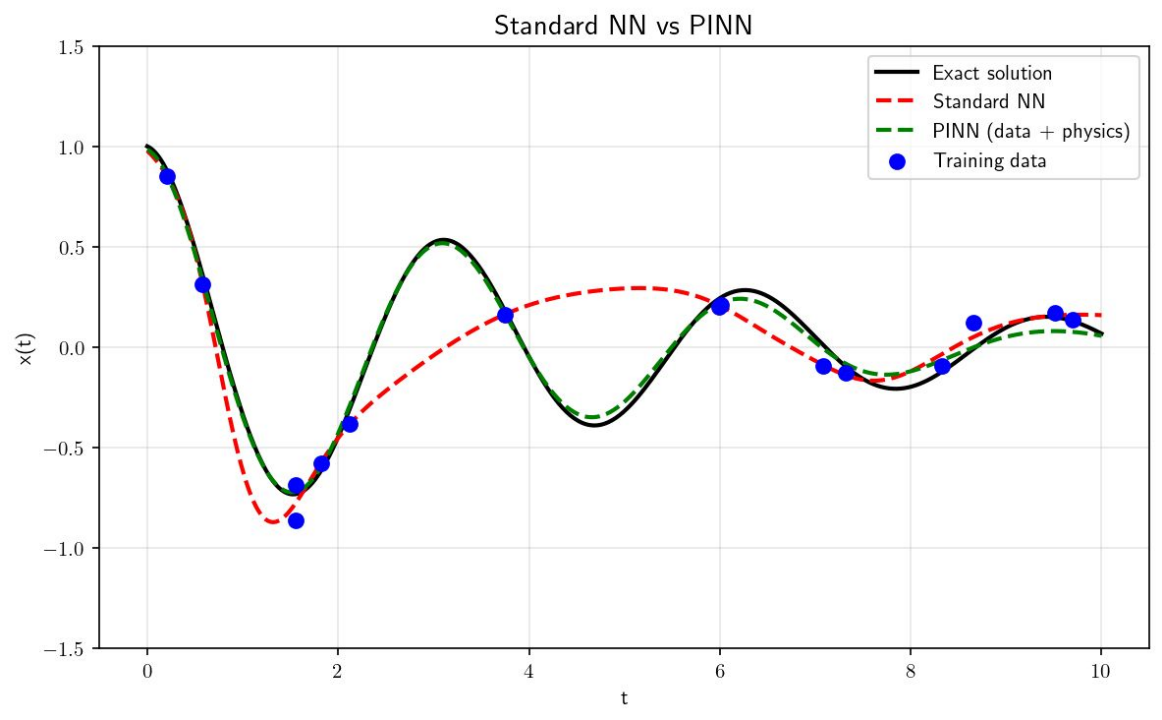
$$\theta''(t) + 2\beta\theta'(t) + \omega_0^2\theta(t) = 0$$



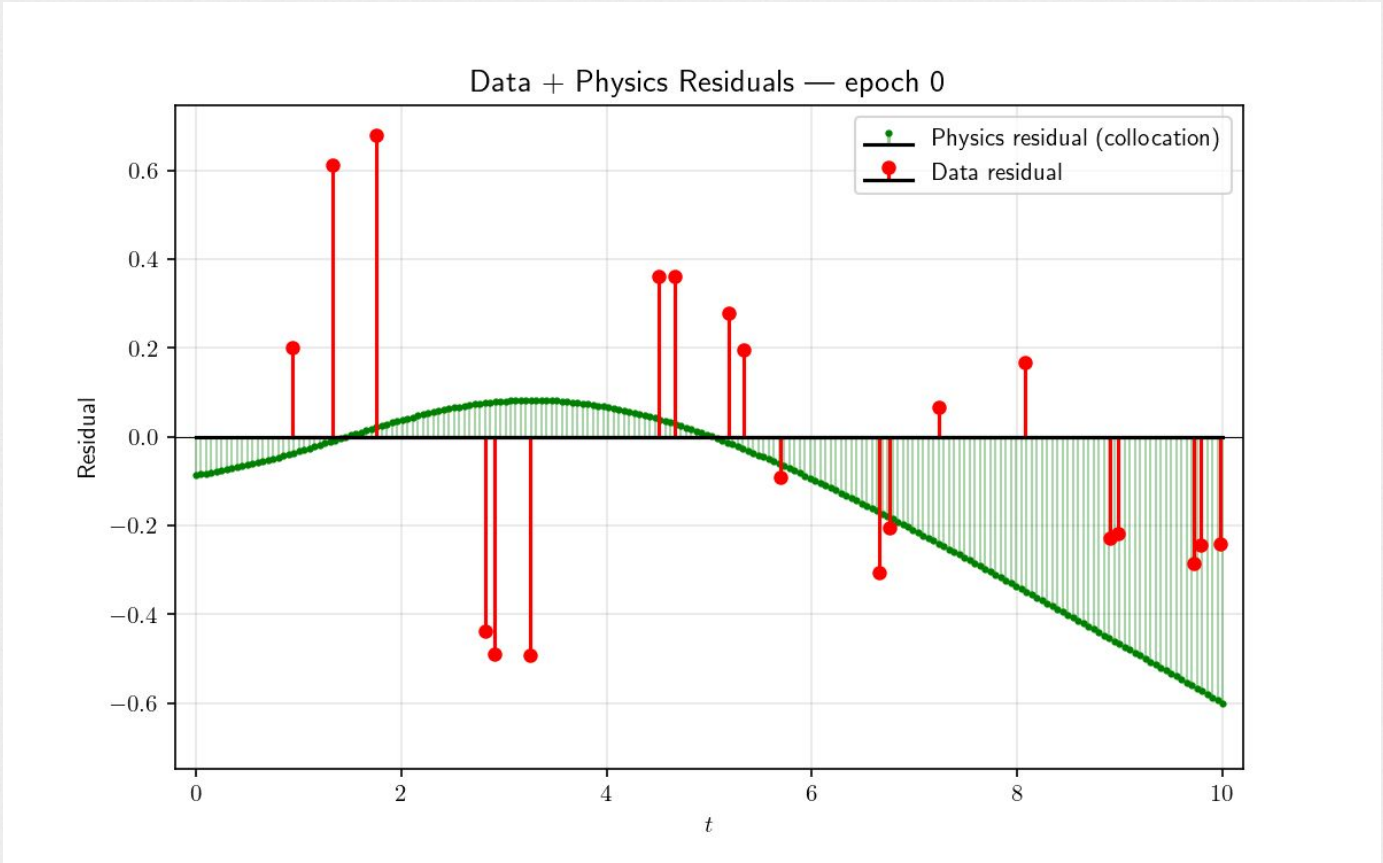
PINNs: Example

- With an added physics loss!

$$\theta''(t) + 2\beta\theta'(t) + \omega_0^2\theta(t) = 0$$



Convergence of the residuals!



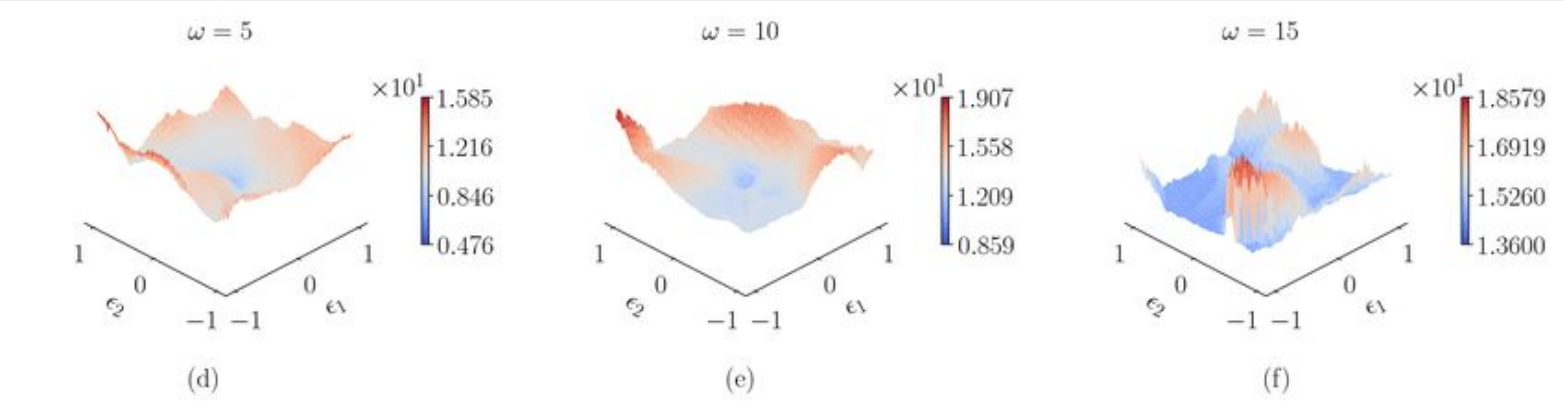
Other strategies

- In this example, we knew the actual differential equation generating the solution. This won't always be the case!
- You can still add various losses to better inform your model. Ex:
  - dE/dt < 0 as our oscillator loses energy.
  - Energy change over time (for our damped oscillator, it forms an approximately exponential decaying envelope!)
- Additionally, with PINNs you can make the parameters learned if they aren't known constants.

PINNs: How does the loss function work?

MACHINE LEARNING @ BERKELEY

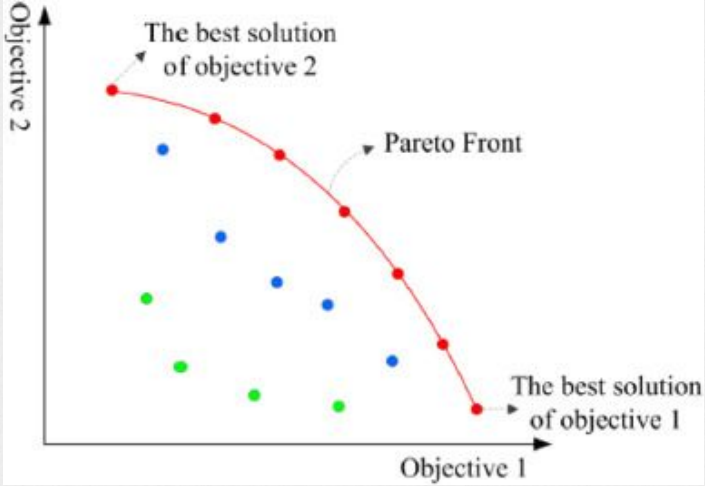
- The beauty of it: Because the physics-informed term does not compute error between a prediction and a target value, this term can be considered an unsupervised loss term
  - collocation points (not just measurements) - any issues with higher dims?

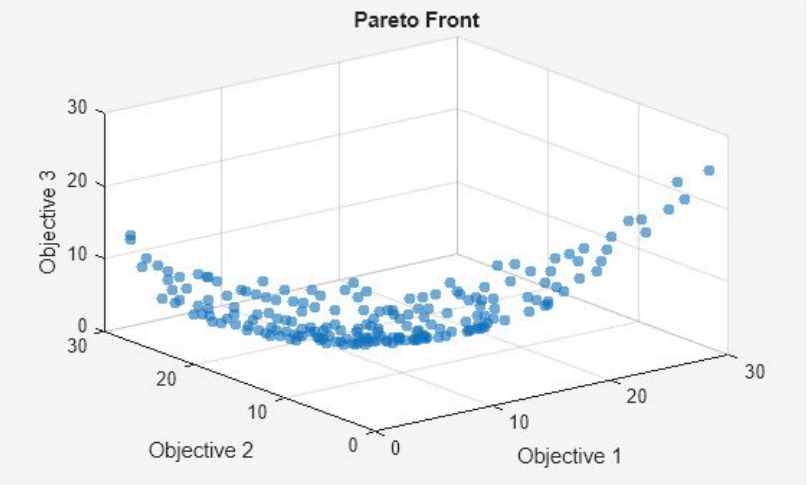


PINNs: Loss function considerations

MACHINE LEARNING @ BERKELEY

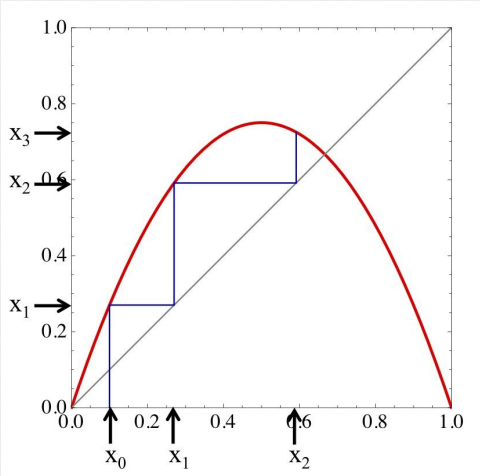
- How do you know terms play nicely together?
  - How big are the magnitudes of the different loss function terms?
  - Weighted loss functions! Use manual tuning/normalization by magnitude
- Multiple objectives - one can override the other

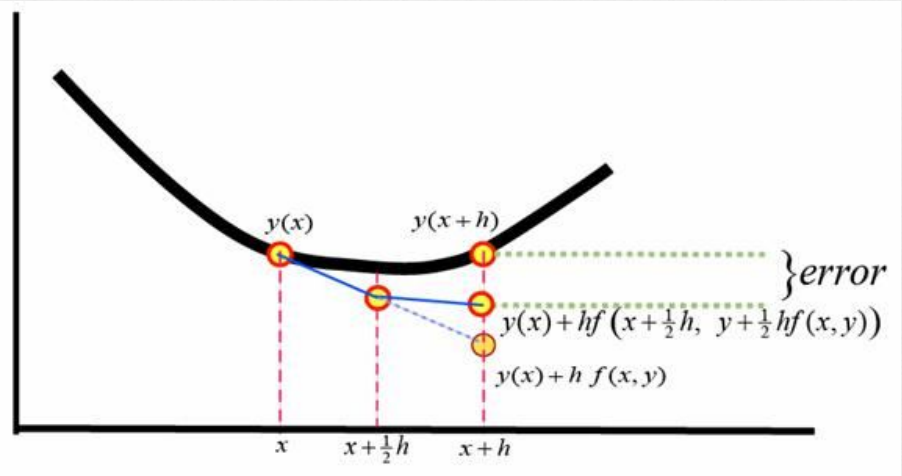




PINNs: Discrete Time Models

- Sequence of time steps, not continuous
- Relies on Runge-Kutta Methods (numerical analysis time-stepping)
  - RK order 4 usually used, evaluate derivative at multiple points and combine
  - Train NNs to step forward in time, learns to output q stages, find weights to satisfy RK constraints -> bigger time steps with higher-order RK

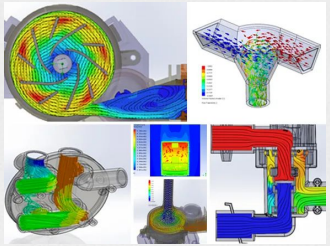




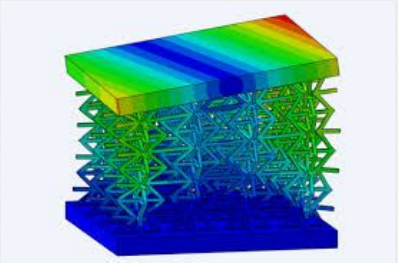
PINNs: Basic Applications

MACHINE LEARNING
@ BERKELEY

- Heat Transfer
  - Equations for thermal processes (heat equation) into loss function
  - Quickly approximate temperature distributions



- Computational Fluid Dynamics
  - Approximating velocity, pressure, and temperature fields of fluids
  - Incorporating Navier-Stokes equations



- Structural mechanics
  - Incorporate equations of elasticity and structural dynamics
  - Predict structural responses like deformations, stresses, and strains

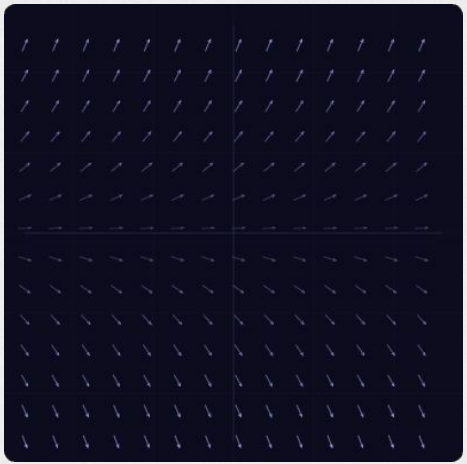
03

Neural ODEs

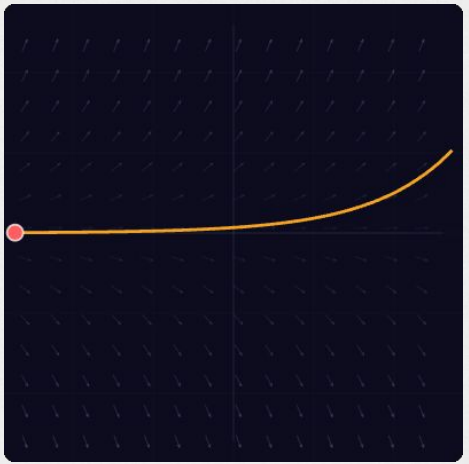
Quick Recap: ODEs

MACHINE LEARNING
@ BERKELEY

- Define a vector field everywhere (what does the tangent line look like at every point), ex: dy/dx = y
- Solution to a general ODE is a family of flows
  - IV constraints this to a unique solution



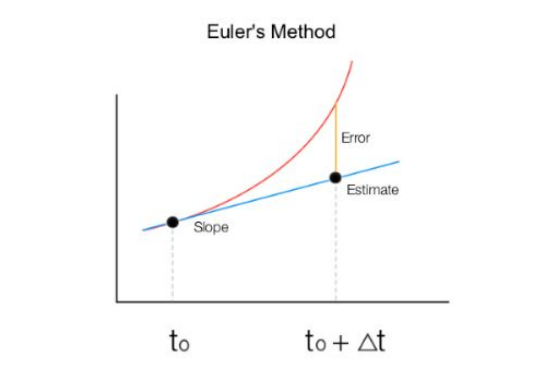


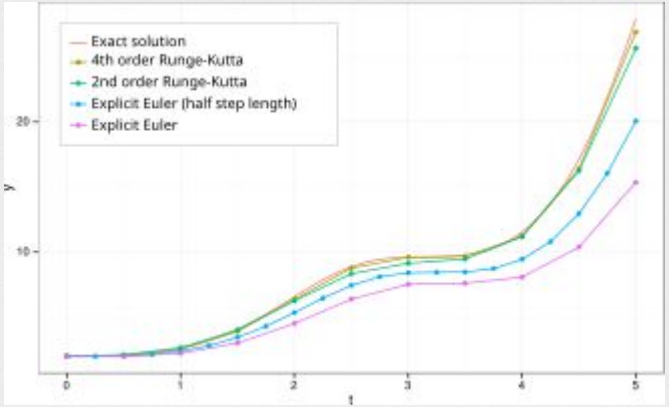


Quick Recap: ODEs

MACHINE LEARNING @ BERKELEY

- How do you actually solve an ODE (how do you find that unique flow?)
  - Closed form solutions: Much easier! Separable solutions, integrating factor (linear ODEs), etc…
    - Solution flow is simply a function like y = Ce^x that can be found
  - Non closed form solutions: More complex ODEs, still has a smooth continuous function solution but we need to use numerical methods (ODE solver)
    - Euler's method, RK4 used to make local approximations based on step size



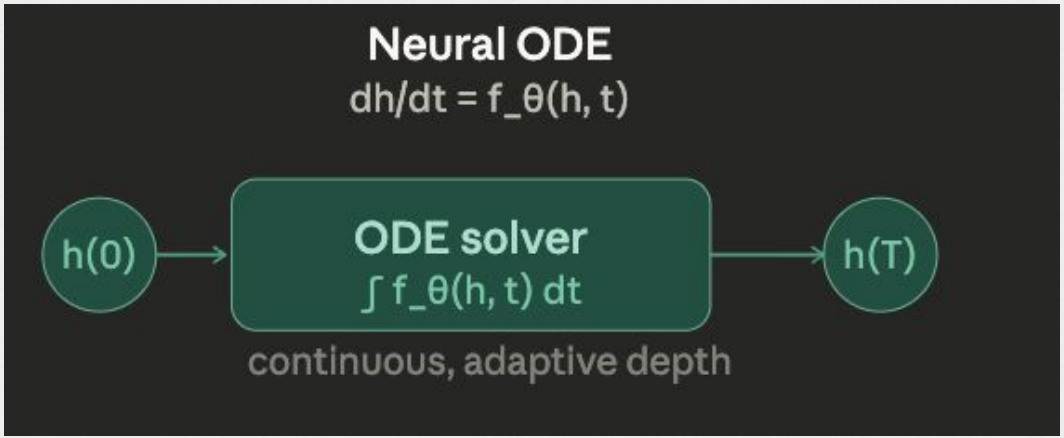


What are Neural ODEs?

MACHINE LEARNING
@ BERKELEY

- Imagine you don't actually have the vector field
  - But you have data!
    - Observed trajectories at discrete time points (t_0, x_0), … (t_n, x_n)

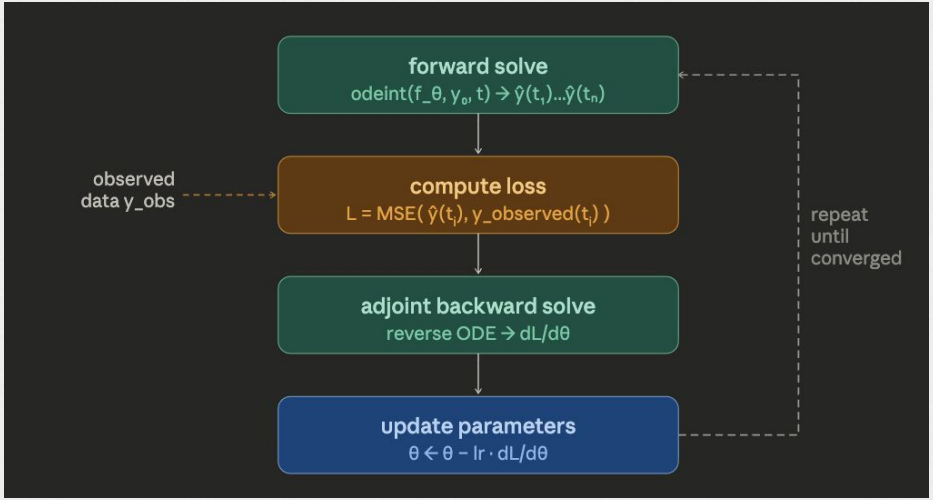
$$\frac{dh}{dt} = f(h, t, \theta)$$



Neural ODEs: Training

MACHINE LEARNING @ BERKELEY

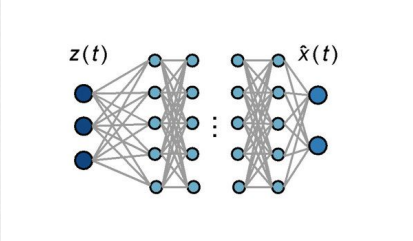
- Training the neural ODE (simple NN):
  - Pick an initial condition x0 and integrate dy/dx = f_θ(y, x) forward
  - MSE between observed x values and predicted x values
    - Backprop through solver - torchdiffeq.odeint vs scipy.integrate.solve_ivp!



Neural ODEs: Advantages and Drawbacks

MACHINE LEARNING
@ BERKELEY

- Advantages:
  - Continuous time representation, naturally handles observations at arbitrary times
  - One small network defines entire dynamics
    - Compared to separate weights per timestep for example

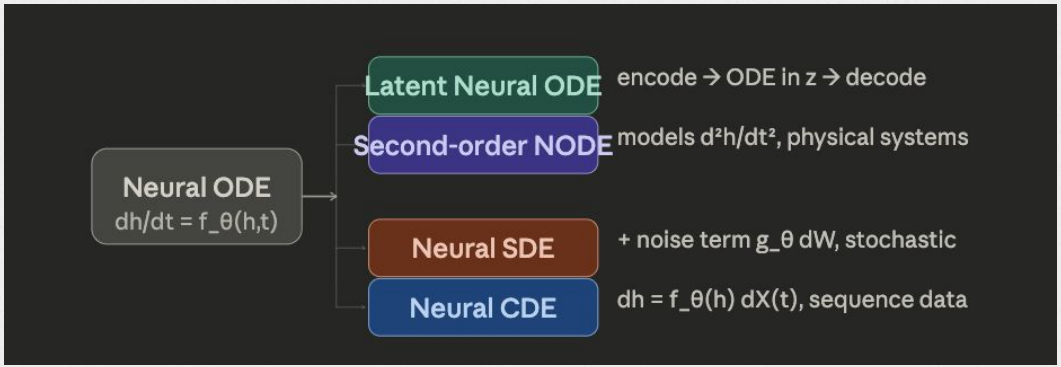


- Disadvantages:
  - Slow training, multiple evaluations of network for every pass
  - Blackbox - don't know what the actual vector field is in the end
  - Sensitive to choice of ODE solver

Neural ODEs: Extensions/variations

MACHINE LEARNING @ BERKELEY

- Latent neural ODEs: run the ODE in a compressed latent space rather than data space
  - Encoder -> ODE evolves -> decoder
- Neural CDEs (Controlled diff. eqs): Dynamics driven by data itself, not just integrating freely after initial IV
  - $dh = f_\theta(h(t))\, dX(t)$, $X(t)$ is continuous interpolation of discrete observations
- Second-order: model d^2h/dt^2 directly rather than reformulating as a first-order system

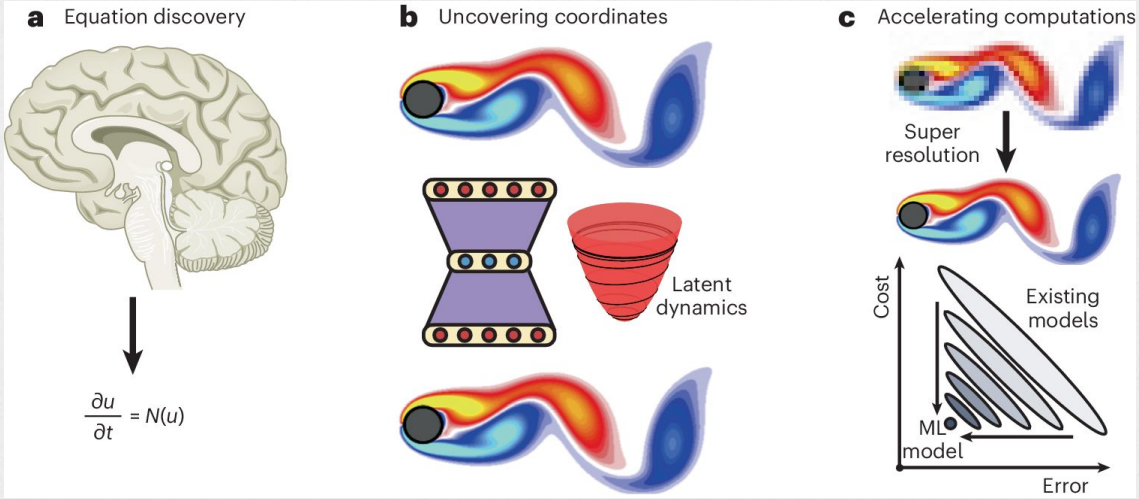


04

Sparse Identification of Nonlinear Dynamics

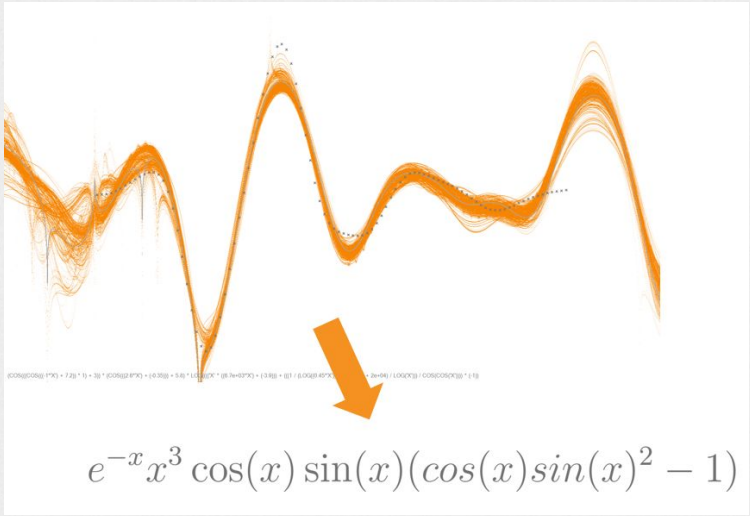
SINDy: Motivation

- Now we know how to embed ODEs into NNs, what about the other direction?
  - What constraints should we apply to the ODEs we find?



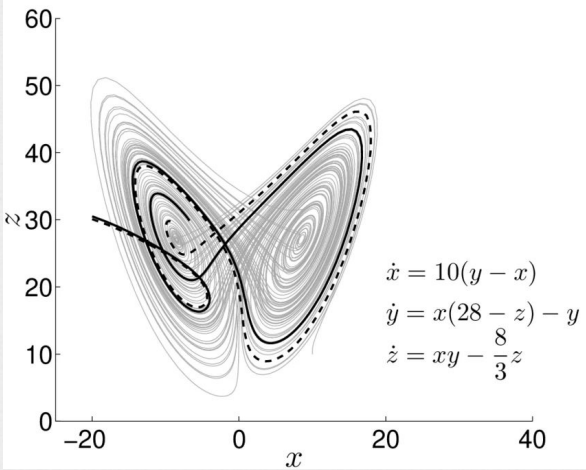
SINDy: Overview

- Most physical systems only have a FEW relevant terms to describe dynamics
- Before: Combinatorial brute-force search - can't scale

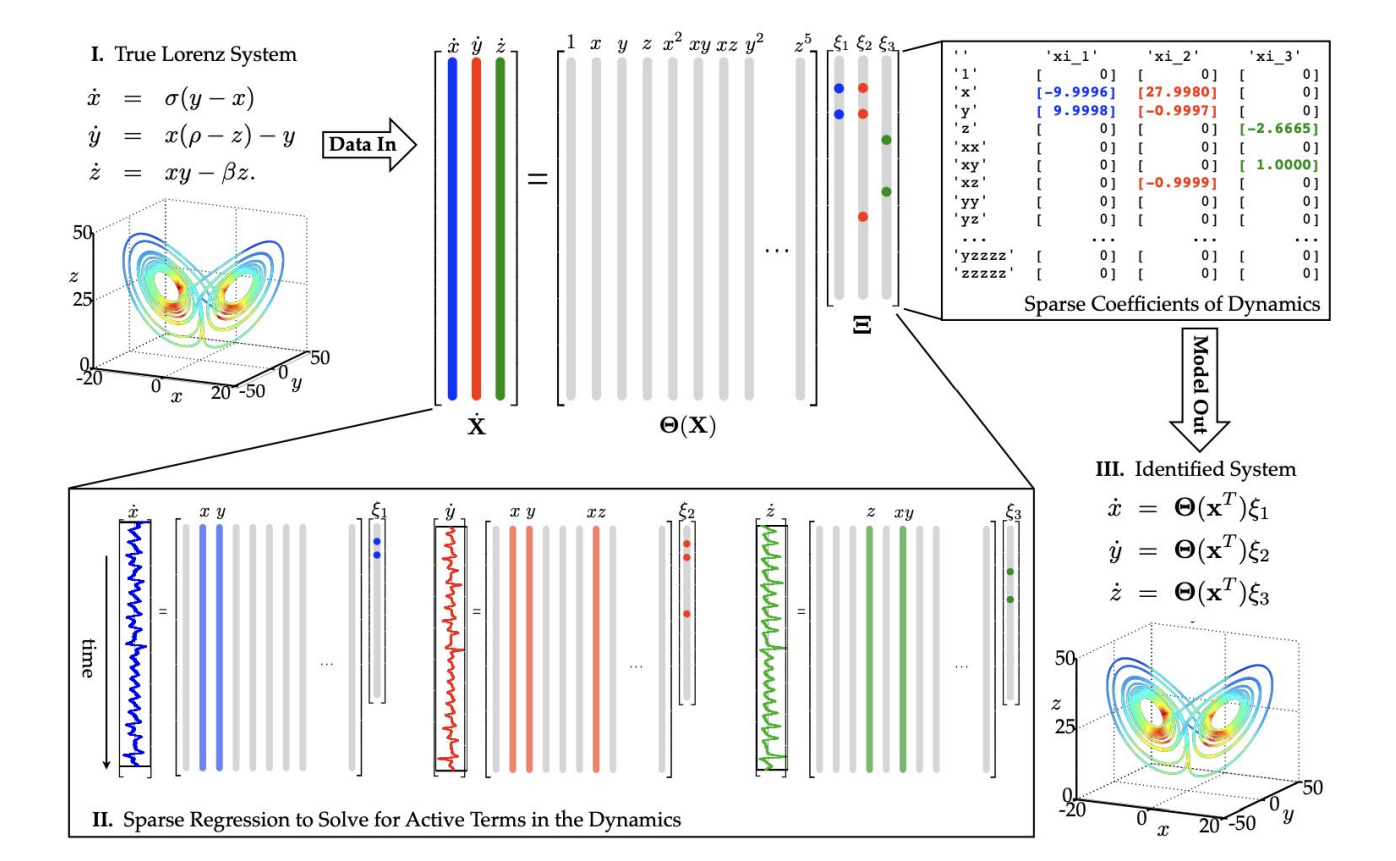


SINDy: Overview

- Symbolic regression: determine function to relate input-output data
  - Alg that builds and tests candidate functions
- Can this be applied to dynamical systems?
  - Overfitting naturally occurs, so parsimony constraint must be imposed



SINDy: Diagram



SINDy: Overview

- How does this extend to governing equations of a physical system?
- Generally, represent nonlinear dynamical system:
  - x(t) is state vector, f(x(t)) represents dynamic constraints

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t)).$$

- To determine a form of f: first collect time-history data

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^T(t_1) \\ \mathbf{x}^T(t_2) \\ \vdots \\ \mathbf{x}^T(t_m) \end{bmatrix} = \overset{\text{state} \rightarrow}{\begin{bmatrix} x_1(t_1) & x_2(t_1) & \cdots & x_n(t_1) \\ x_1(t_2) & x_2(t_2) & \cdots & x_n(t_2) \\ \vdots & \vdots & \ddots & \vdots \\ x_1(t_m) & x_2(t_m) & \cdots & x_n(t_m) \end{bmatrix}} \Big\downarrow \text{time}$$

$$\dot{\mathbf{X}} = \begin{bmatrix} \dot{\mathbf{x}}^T(t_1) \\ \dot{\mathbf{x}}^T(t_2) \\ \vdots \\ \dot{\mathbf{x}}^T(t_m) \end{bmatrix} = \begin{bmatrix} \dot{x}_1(t_1) & \dot{x}_2(t_1) & \cdots & \dot{x}_n(t_1) \\ \dot{x}_1(t_2) & \dot{x}_2(t_2) & \cdots & \dot{x}_n(t_2) \\ \vdots & \vdots & \ddots & \vdots \\ \dot{x}_1(t_m) & \dot{x}_2(t_m) & \cdots & \dot{x}_n(t_m) \end{bmatrix}.$$

SINDy: Algorithm

- Construct augmented library of candidate terms!

$$\Theta(\mathbf{X}) = \begin{bmatrix} | & | & | & | & & | & | & | & | & \\ \mathbf{1} & \mathbf{X} & \mathbf{X}^{P_2} & \mathbf{X}^{P_3} & \cdots & \sin(\mathbf{X}) & \cos(\mathbf{X}) & \sin(2\mathbf{X}) & \cos(2\mathbf{X}) & \cdots \\ | & | & | & | & & | & | & | & | & \end{bmatrix}$$

$$\mathbf{X}^{P_2} = \begin{bmatrix} x_1^2(t_1) & x_1(t_1)x_2(t_1) & \cdots & x_2^2(t_1) & x_2(t_1)x_3(t_1) & \cdots & x_n^2(t_1) \\ x_1^2(t_2) & x_1(t_2)x_2(t_2) & \cdots & x_2^2(t_2) & x_2(t_2)x_3(t_2) & \cdots & x_n^2(t_2) \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ x_1^2(t_m) & x_1(t_m)x_2(t_m) & \cdots & x_2^2(t_m) & x_2(t_m)x_3(t_m) & \cdots & x_n^2(t_m) \end{bmatrix}$$

SINDy: Overview

- Back to the fundamentals: where have we seen sparsity constraints?
  - LASSO!

$$\mathbf{y} = \Theta\boldsymbol{\xi}.$$

$$\boldsymbol{\xi} = \underset{\boldsymbol{\xi}'}{\operatorname{argmin}} \|\Theta\boldsymbol{\xi}' - \mathbf{y}\|_2 + \lambda\|\boldsymbol{\xi}'\|_1.$$

SINDy: Algorithm

- Each column of Ξ represents sparse vector of coefficients determining which terms are active - (understand distinction between X and x)!

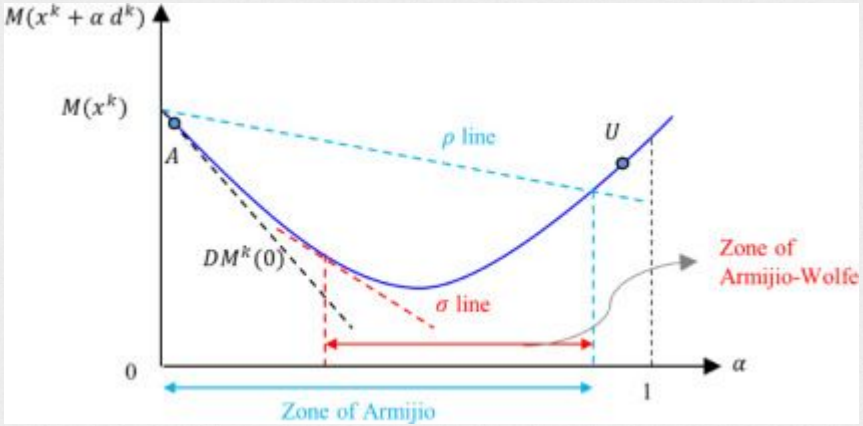
$$\dot{\mathbf{X}} = \mathbf{\Theta}(\mathbf{X})\mathbf{\Xi}.$$

$$\dot{\mathbf{x}}_k = \mathbf{f}_k(\mathbf{x}) = \mathbf{\Theta}(\mathbf{x}^T)\boldsymbol{\xi}_k.$$

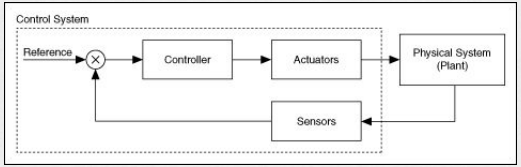
SINDy: Solve for coefficients

$$\dot{\mathbf{X}} = \mathbf{\Theta}(\mathbf{X})\mathbf{\Xi} + \eta\mathbf{Z},$$

- Include noise!
  - LASSO works but may be computationally expensive…
- Use sequential thresholded least-squares algorithm
  - Do regular least squares, then apply some cutoff for coefficients and repeat



SINDy: Extensions



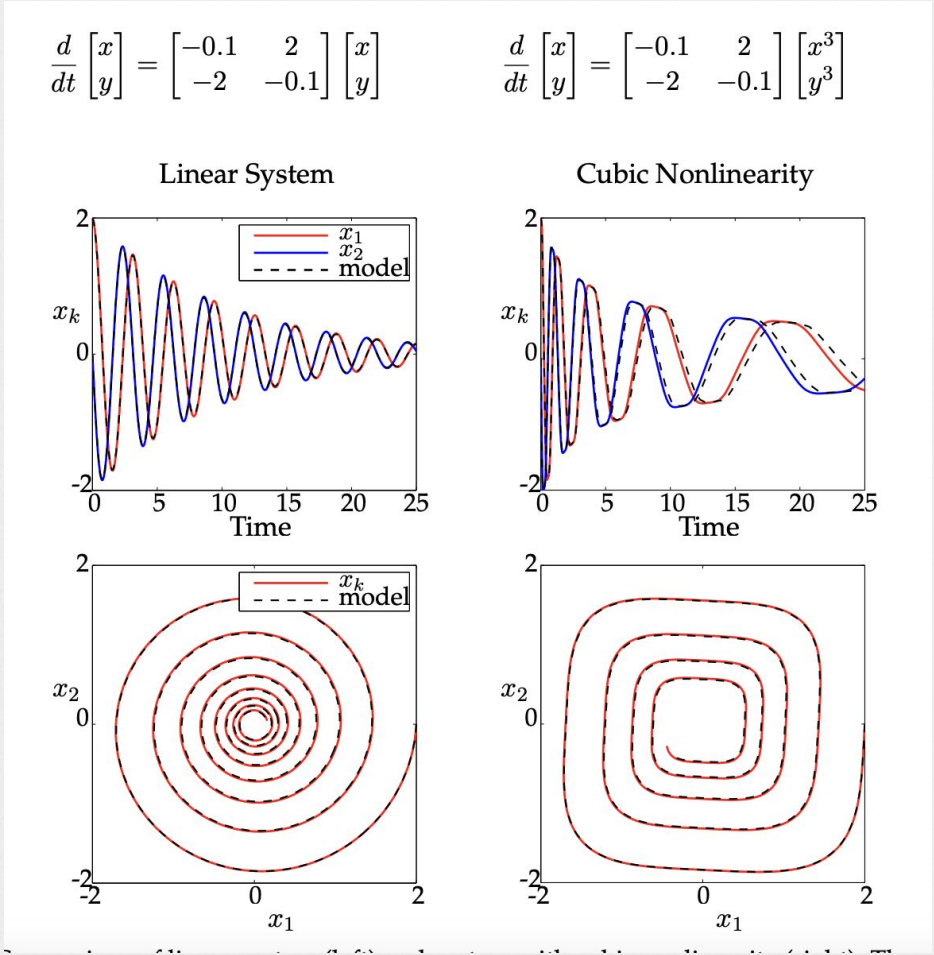
- Bifurcation parameters, external forcing
  - Sparse combination of components in x and bifurcation parameter (ex damping)
  - External forcing can also be added
    - Ex: feedback control (regulations, taxes, interest in financial systems)

$$\begin{aligned} \dot{\mathbf{x}} &= \mathbf{f}(\mathbf{x}, \mathbf{u}(t), t) \\ \dot{t} &= 1. \end{aligned}$$

$$\begin{aligned} \dot{\mathbf{x}} &= \mathbf{f}(\mathbf{x}; \boldsymbol{\mu}) \\ \dot{\boldsymbol{\mu}} &= \mathbf{0}. \end{aligned}$$

SINDy: Example

- Damped oscillator example
  - Cubic - nonlinear spring



SINDy: Example

	xdot	ydot
1	0	0
x	-0.1015	-1.9990
y	2.0027	-0.0994
xx	0	0
xy	0	0
yy	0	0
xxx	0	0
xxy	0	0
xyy	0	0
yyy	0	0
xxxx	0	0
xxxy	0	0
xxyy	0	0
xyyy	0	0
yyyy	0	0
xxxxx	0	0
xxxxy	0	0
xxxyy	0	0
xxyyy	0	0
xyyyy	0	0
yyyyy	0	0

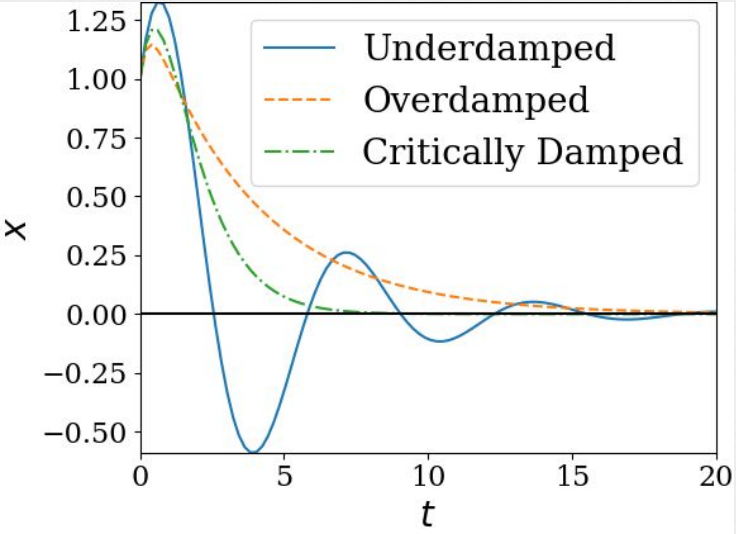
	xdot	ydot
1	0	0
x	0	0
y	0	0
xx	0	0
xy	0	0
yy	0	0
xxx	-0.0996	-1.9994
xxy	0	0
xyy	0	0
yyy	1.9970	-0.0979
xxxx	0	0
xxxy	0	0
xxyy	0	0
xyyy	0	0
yyyy	0	0
xxxxx	0	0
xxxxy	0	0
xxxyy	0	0
xxyyy	0	0
xyyyy	0	0
yyyyy	0	0

05

Fourier Neural Operators

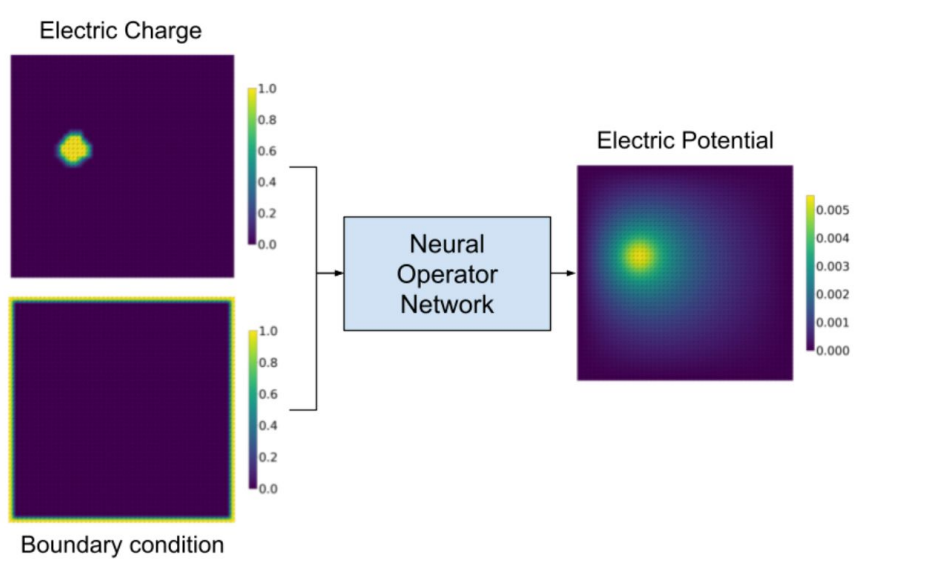
Neural Operators: Motivation

- Recall PINNs - we can get individual solutions
  - Once trained, gives e.g. oscillator position given particular timestep
- Can we be more general?
  - Get e.g. oscillator's entire trajectory given initial position and velocity?
  - Would allow us to use one NN for many different similar models



Neural Operators: Motivation

- Operator: maps function to function
- PINNs can give solutions to specific cases, but we want to generalize
- Can we learn a **general solution operator** from PDE inputs to outputs?

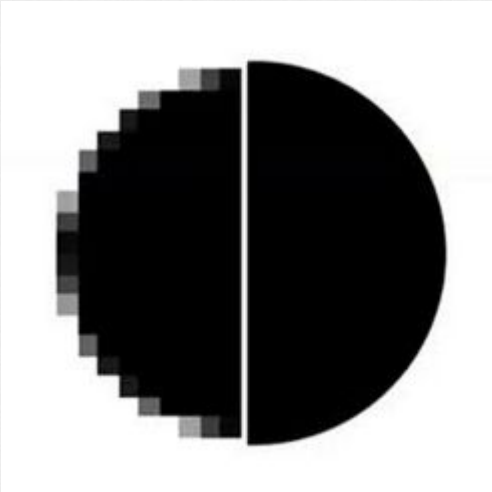


Neural Operators: Overview

Key difference: continuous domain, not discrete domain

➔ We want to generate entire trajectory, not one at specific times

➔ **Convolution** of layers instead of a FC layer



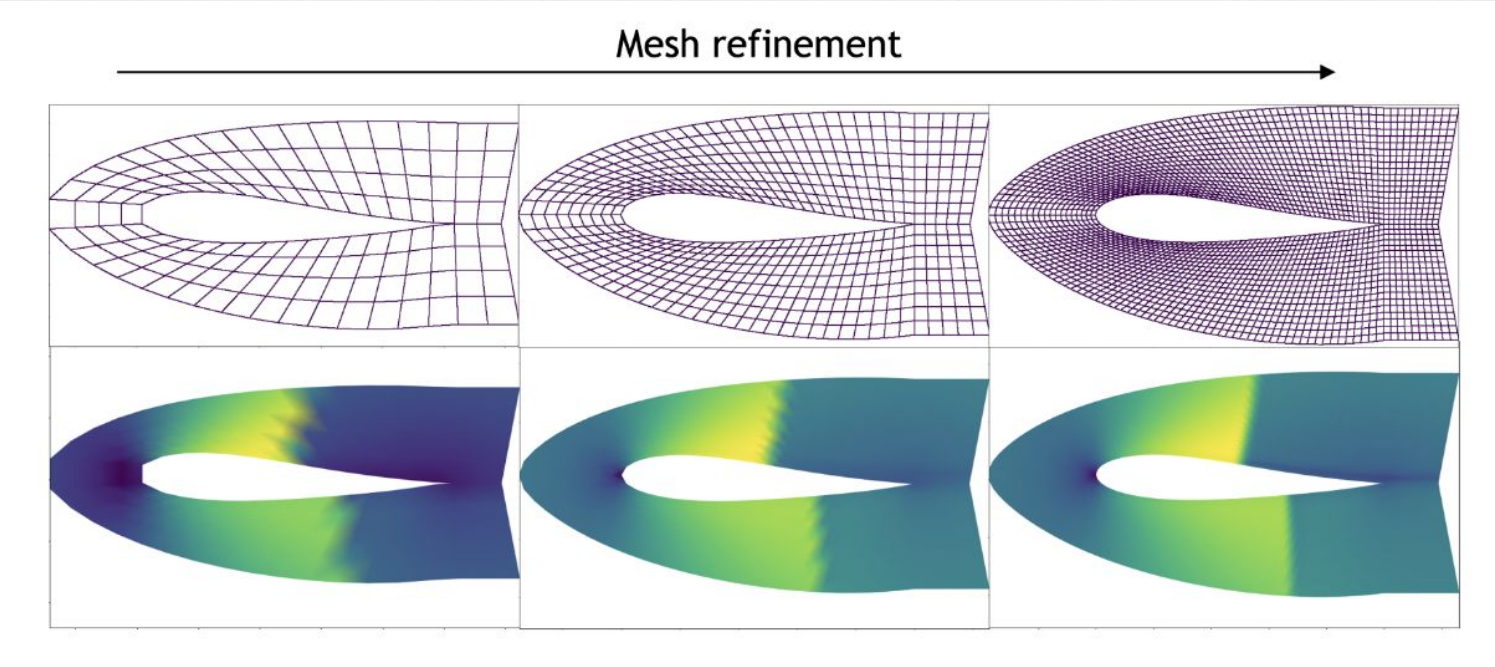
$$u(x) = \int_D G_a(x,y)[f(y) + (\Gamma_a u)(y)]\, dy.$$

$$\int_D \kappa_\phi(x, y, a(x), a(y)) v_t(y)\, \nu_x(dy)$$

$$u = Q\left(K_l \circ \sigma_l \circ \cdots \circ \sigma_1 \circ K_0\right) P\, v$$

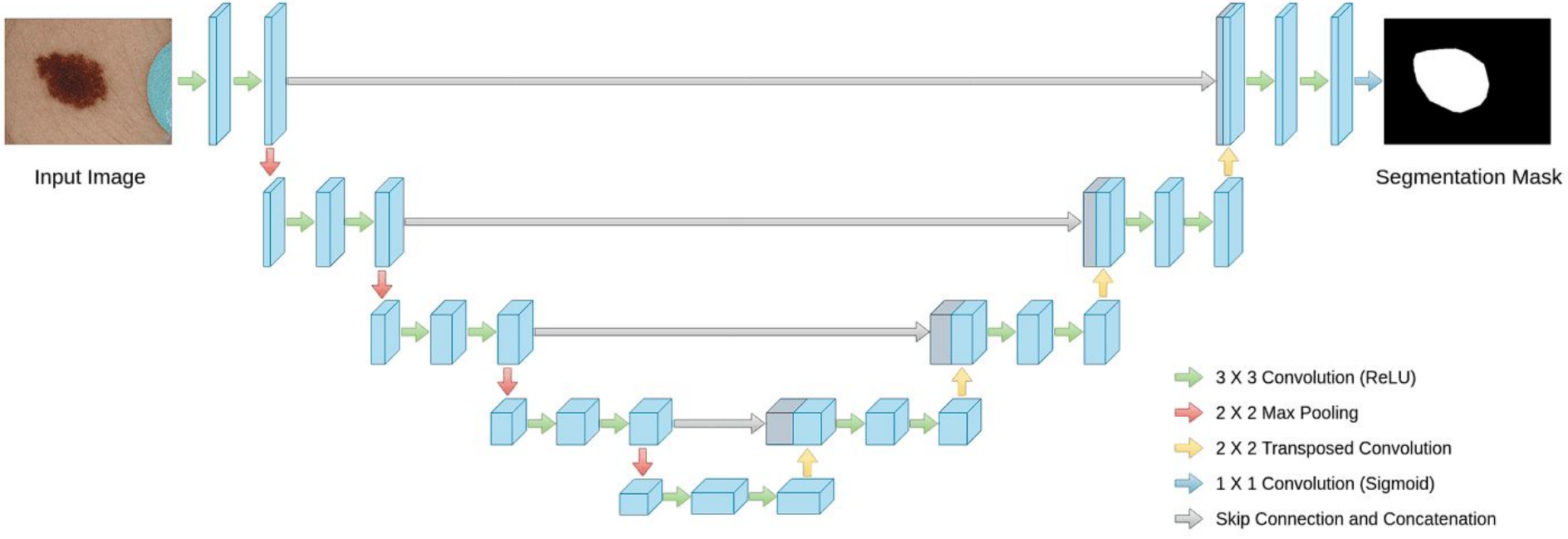
Neural Operators: Benefits

- Doesn’t depend on specific choice of points!



Neural Operators: Intuition

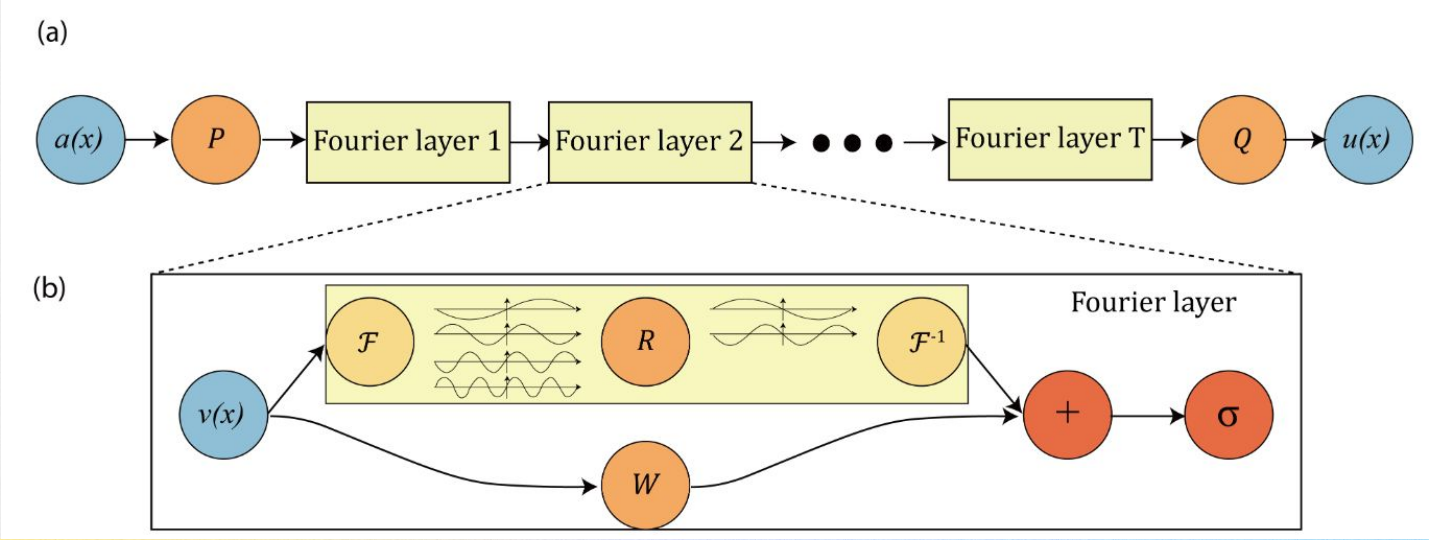
- Image-to-image problem



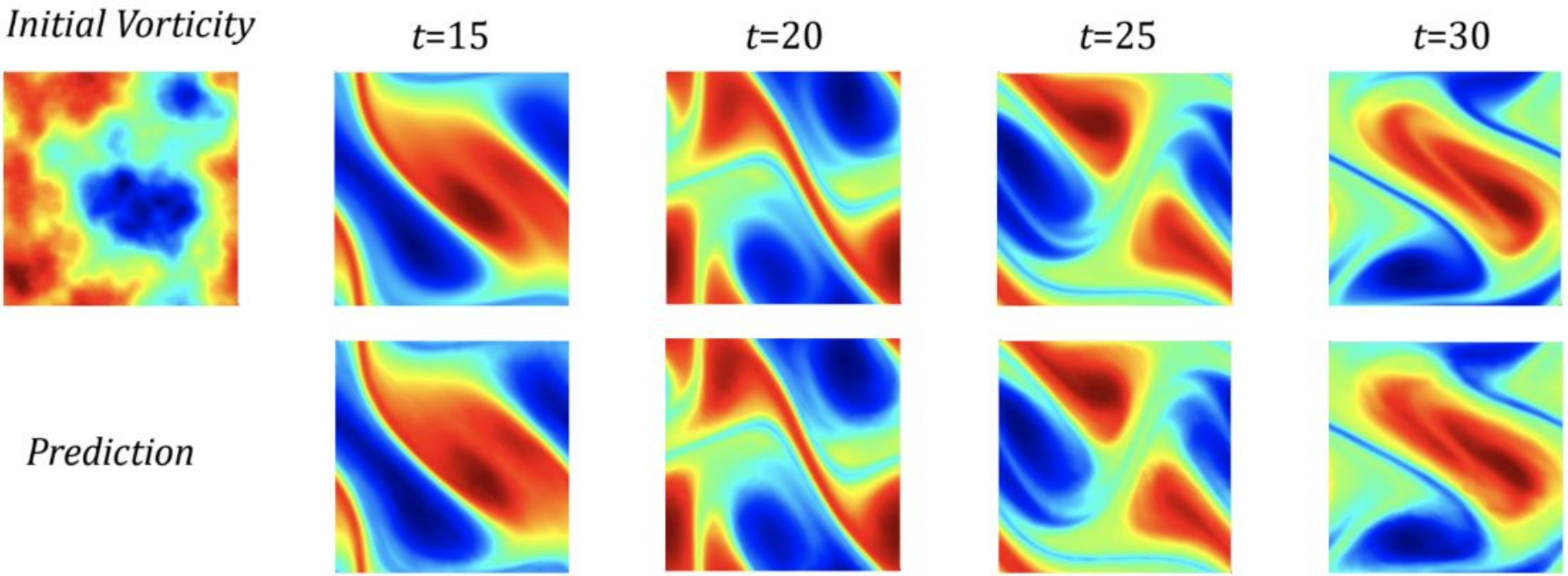
Fourier Neural Operators: What's different?

- Special property of spectral domain: convolutions become multiplications

$$\big(\mathcal{K}(a;\phi)v_t\big)(x) := \int_D \kappa\big(x, y, a(x), a(y); \phi\big) v_t(y) \mathrm{d}y,$$



FNOs: Results

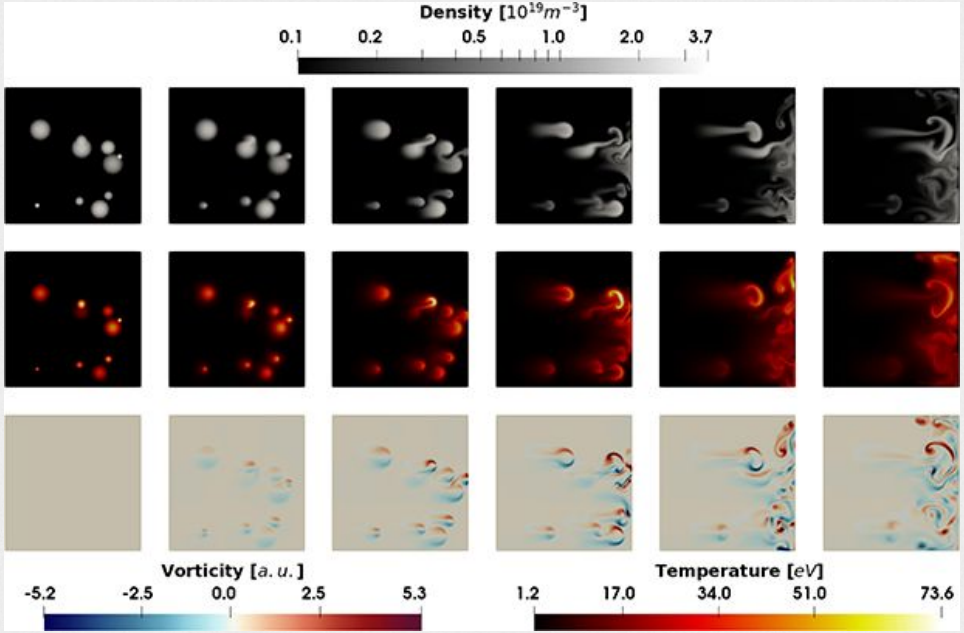


FNOs: Results

Config	Parameters	Time per epoch	$\nu = 1e{-}3$ $T = 50$ $N = 1000$	$\nu = 1e{-}4$ $T = 30$ $N = 1000$	$\nu = 1e{-}4$ $T = 30$ $N = 10000$	$\nu = 1e{-}5$ $T = 20$ $N = 1000$
FNO-3D	$6,558,537$	$38.99s$	**0.0086**	0.1918	**0.0820**	0.1893
FNO-2D	$414,517$	$127.80s$	0.0128	**0.1559**	0.0834	**0.1556**
U-Net	$24,950,491$	$48.67s$	0.0245	0.2051	0.1190	0.1982
TF-Net	$7,451,724$	$47.21s$	0.0225	0.2253	0.1168	0.2268
ResNet	$266,641$	$78.47s$	0.0701	0.2871	0.2311	0.2753

Neural Operators: Applications

- Fluid and solid mechanics
  - Turbulent Flow!!!
  - Weather forecasts
- Nuclear fusion
- ML research
  - Diffusion and ViTs
  - Traditional CV tasks w/ resolution invariance

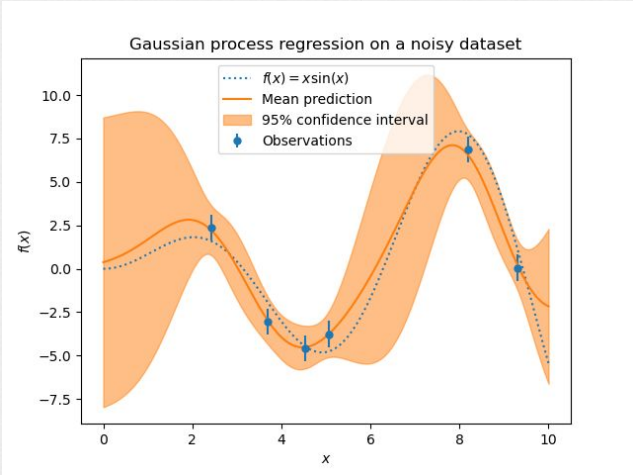


06

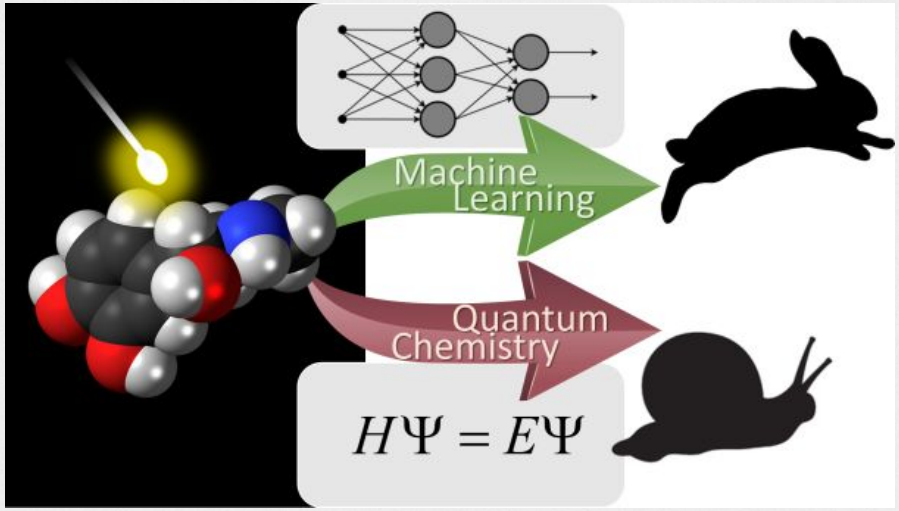
Applications of Physics ML

Gaussian Processes - Uncertainty!!

- GP defined by mean and kernel (how function values are correlated)
- Define distribution over all possible functions, use data to narrow down functions
  - Enforce physical constraints in kernel (ex wave equation)



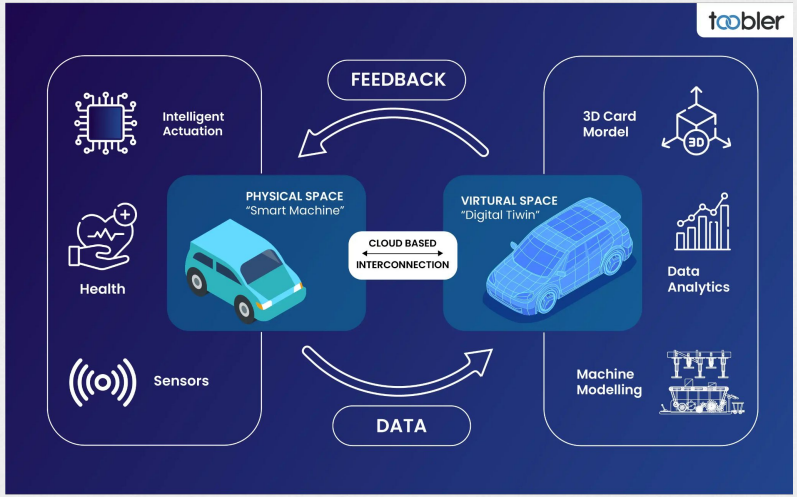
Computational Chemistry

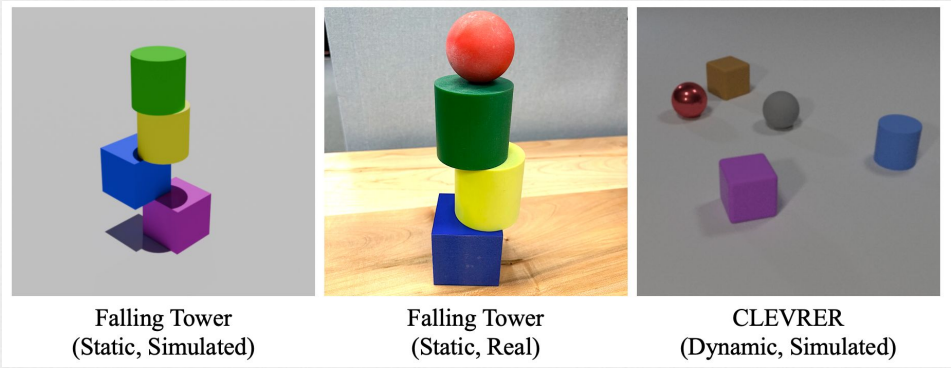


- GNNs are extremely powerful in modeling chemical systems
- Using ML potentials for simulations can give 1000x speedup or more
- Enforcing rotational/translational symmetries and incorporating magnetic interactions between atoms increases quality of potentials

Robotics

- Physics engines are used in robotics simulators to generate realistic training data
- Enforcing physical reasoning in VLMs can increase performance



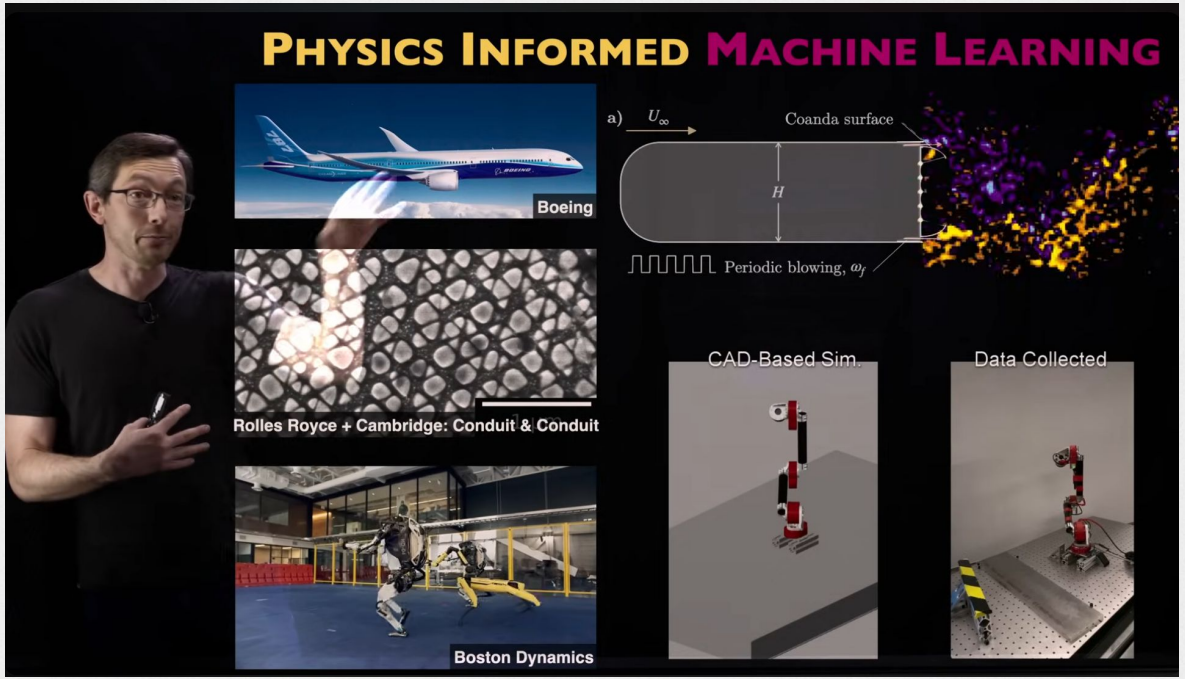


07

Get Involved!

STEVE BRUNTON IS THE GOAT





Want to learn more?





MACHINE LEARNING
@ BERKELEY

Title of Presentation

Presenters & Date



MACHINE LEARNING
@ BERKELEY



MACHINE LEARNING
@ BERKELEY



MACHINE LEARNING
@ BERKELEY

Title of Presentation

Presenters & Date



MACHINE LEARNING
@ BERKELEY



MACHINE LEARNING
@ BERKELEY

AGENDA

01 First Point
02 Second Point
03 Third Point
04 Fourth Point

ABOUT



First and Last □ Role on the team □ Lorem ipsum dolor sit amet rolod



First and Last □ Role on the team □ Lorem ipsum dolor sit amet rolod



First and Last □ Role on the team □ Lorem ipsum dolor sit amet rolod



First and Last □ Role on the team □ Lorem ipsum dolor sit amet rolod



First and Last □ Role on the team □ Lorem ipsum dolor sit amet rolod



First and Last □ Role on the team □ Lorem ipsum dolor sit amet rolod



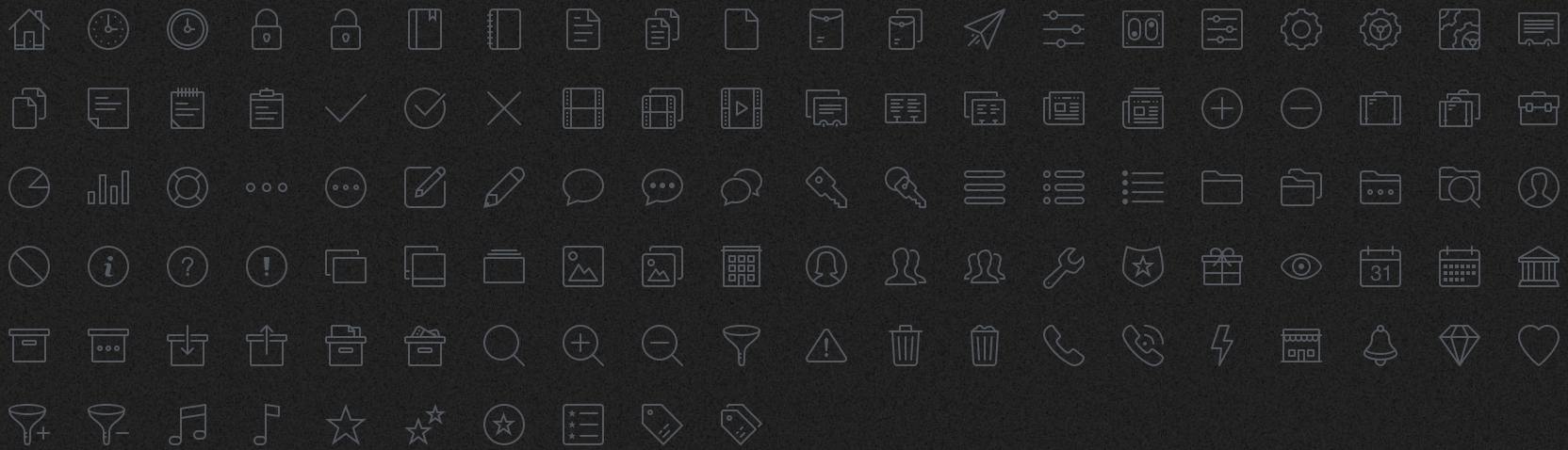
MACHINE LEARNING
@ BERKELEY

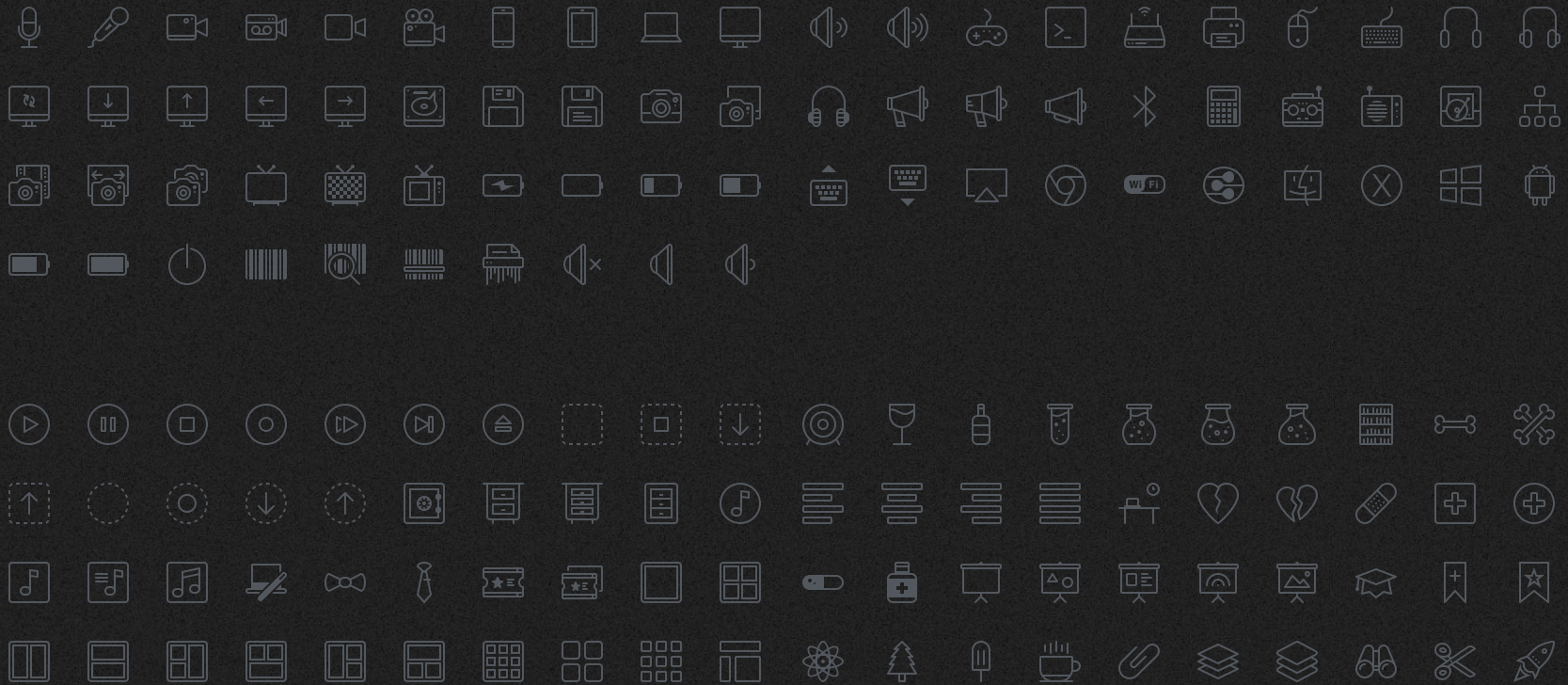


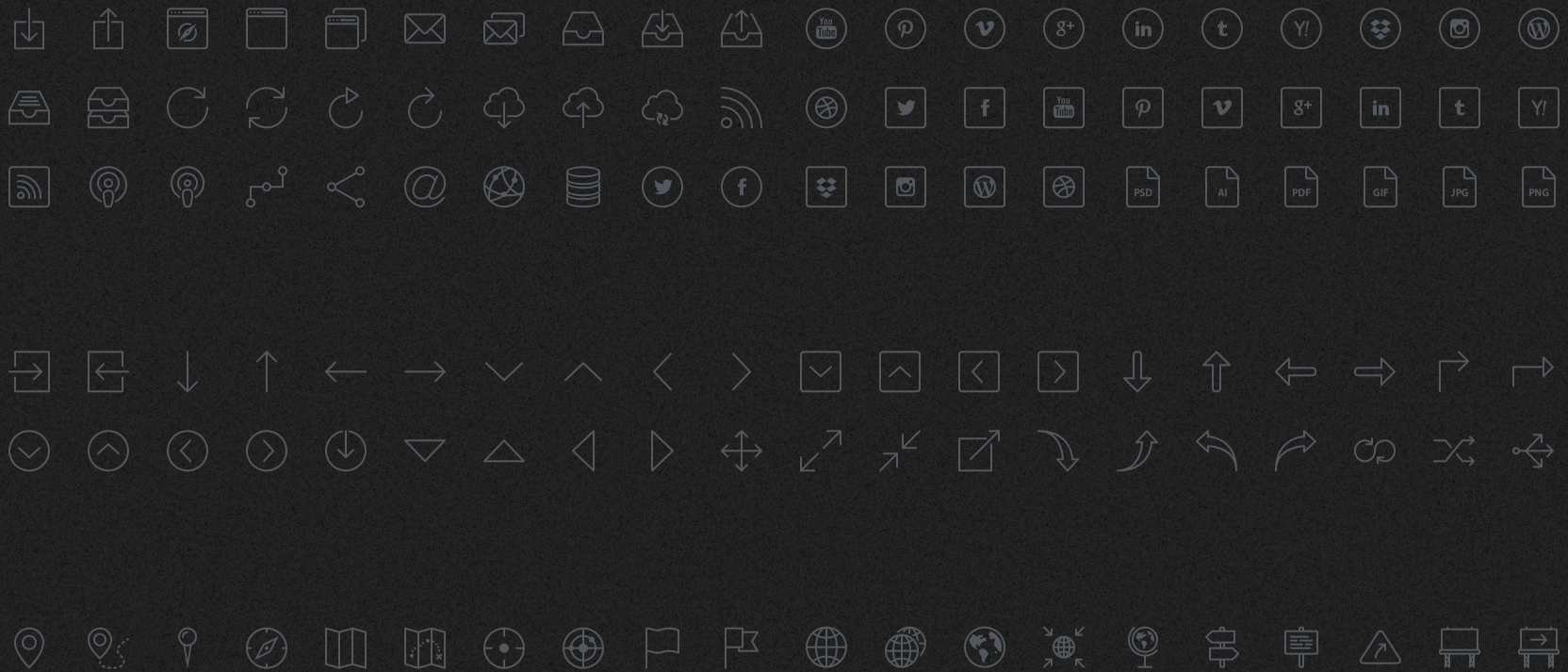
MACHINE LEARNING
@ BERKELEY



MACHINE LEARNING
@ BERKELEY






PSD
AI
PDF
GIF
JPG
PNG